

DRL course summary

Tamir Levin

February 2026

Contents

1	Lecture 1: Introduction to Deep Reinforcement Learning	3
1.1	Foundations of Reinforcement Learning	3
1.2	Planning and Model Assumptions	5
1.3	Policy & Exploration	5
2	Lecture 2: Deep Q-Learning, Policy Gradients, and Actor-Critic Methods	8
2.1	Deep Q-Learning (DQN)	8
2.2	Value-Based Methods (Advanced DQN Architectures)	9
2.3	Policy Gradient Methods	10
2.4	Actor-Critic Methods	12
3	Lecture 3: Imitation Learning	12
3.1	Regret in RL Algorithms	13
3.2	Introduction to Imitation Learning	13
3.3	Imitation Learning Algorithms	13
4	Lecture 4: Multi-Arm Bandits	15
4.1	The Multi-Arm Bandit (MAB) Problem	15
4.2	Simple Bandit Algorithms (Context-Free)	16
4.3	Advanced Bandit Algorithms	17
5	Lecture 5: Monte Carlo Tree Search, AlphaGo, and AutoML	18
5.1	Monte Carlo Tree Search (MCTS)	18
5.2	Reinforcement Learning in the Game of Go	19
5.3	Applications in AutoML	21
6	Lecture 6: Meta-Learning and Transfer Learning	22
6.1	Introduction to Meta-Learning	22
6.2	Meta-Learning with Memory-Augmented Neural Networks (MANN)	22
6.3	Simple Neural Attentive Meta-Learner (SNAIL)	23
6.4	Model-Agnostic Meta-Learning (MAML)	23
6.5	Real-Time Adaptation	24
6.6	Forward Transfer Learning	26
6.7	Multi-Task Transfer Learning	27
7	Lecture 7: Dealing with Large State and Action Spaces	28
7.1	Large Discrete Action Spaces	28
7.2	The Wolpertinger Policy	28
7.3	Generative Proto-Sequence (GPS)	28
7.4	Action Elimination (AE-DQN)	29
7.5	Hierarchical Reinforcement Learning (HRL)	30
7.6	Advanced Scaling Architectures	31

8	Lecture 8: Advanced Model Learning and Exploration	32
8.1	Learning in Latent Space	32
8.2	Generative Models and Variational Learning	33
8.3	Embed to Control (E2C)	34
8.4	Action-Conditional Video Prediction	35
8.5	Informed Exploration	36
8.6	Inverse Reinforcement Learning on Images	36
8.7	Exemplar Models for Exploration (EX ²)	37
8.8	Curiosity-Driven Exploration (ICM)	38
9	Lecture 9: Transformers in Deep Reinforcement Learning	39
9.1	Introduction to Transformers	39
9.2	BERT: Bidirectional Representations	39
9.3	Challenges of Transformers in DRL	40
9.4	Gated Transformer-XL (GTrXL)	40
9.5	Handling Long Sequences	41
9.6	Trajectory Transformer	42
9.7	Decision Transformer (DT)	42
9.8	Elastic Decision Transformer (EDT)	43
9.9	Online Decision Transformer (ODT)	43
9.10	Safe and Multi-Game Transformers	44
9.11	Feedback Decision Transformer (FDT)	45
10	Lecture 10: Model-Based Approaches and Gradient-Free Learning	46
10.1	Introduction to Model-Based Learning	46
10.2	Evolution of Model-Based Learning Algorithms	47
10.3	Global vs. Local Models	48
10.4	Linear Quadratic Regulator (LQR)	49
10.5	Model Predictive Control (MPC)	49
10.6	Guided Policy Search (GPS)	50
10.7	Gradient-Free Methods and Evolutionary Algorithms	50
10.8	Covariance Matrix Adaptation Evolution Strategy (CMA-ES)	50
11	Lecture 11: World Models in Deep Reinforcement Learning	50
11.1	Introduction to World Models	50
11.2	Dimension 1: Latent Representation Types	52
11.3	Dimension 2: Dynamics Formulation	53
11.4	Dimension 3: Planning and Behavior Learning Strategy	54
11.5	Dimension 4: World Model Training Objectives	56
11.6	PlaNet (Deep Planning Network)	57
11.7	Dreamer	58

1 Lecture 1: Introduction to Deep Reinforcement Learning

1.1 Foundations of Reinforcement Learning

1.1.1 The Necessity of Reinforcement Learning

Reinforcement Learning (RL) is defined as a semi-supervised learning approach where an agent interacts with an environment to maximize rewards. While supervised learning is excellent for static classification, RL is designed for decision sequences.

A practical example provided in the lecture is malware detection. Current platforms often use an ensemble of multiple detectors, which creates significant redundancy and is computationally expensive. Instead of querying every detector, a DRL agent can be trained to query a subset of detectors and decide, based on initial classifications, whether to query more. This requires modeling both the "reward" of a correct classification and the "cost" of the computation.

1.1.2 Reinforcement Learning vs. Supervised Learning

The fundamental distinction between DRL and supervised learning lies in the nature of the environment. In DRL, the environment changes based on the actions taken by the actor, whereas in supervised learning, decisions are generally made in a static setting. There are three primary technical challenges unique to RL:

1. The agent requires constant access to the environment to learn, which is simple in games like Atari but difficult in real-world scenarios like self-driving.
2. The agent must learn and plan from correlated samples, as following states in a trajectory are highly dependent on one another.
3. The data distribution itself changes over time based on the actions the agent chooses to take.

Successful historical applications include TD-Gammon in the 1990s, which beat human experts in Backgammon (Shesh-Besh), and AlphaGo in 2016. RL is the best solution when data comes in the form of trajectories, a sequence of related decisions is required, or feedback to actions is partial and noisy. It overcomes classic DL/ML solutions when a sacrifice is needed now in order to get a higher reward later.

1.1.3 Markov Decision Processes (MDP)

The MDP framework provides a mathematical structure for RL environments. It consists of States (S) which represent the current conditions or circumstances, Actions (A) which are the transitions between states, and Rewards (R) which are scalar values produced by taking a specific action in a state. The core of this framework is the **Markov Property**, which states that the future depends only on the present state and action, not on the previous history:

$$Pr(s_{t+1}|s_1, a_1, \dots, s_t, a_t) = Pr(s_{t+1}|s_t, a_t) \quad (1)$$

This property simplifies the problem significantly, allowing the calculation of transition probabilities $p(s', r|s, a)$ without needing to track every past move.

1.1.4 Returns

The agent's objective is to maximize the expected return. In a finite horizon, the return G_t is the sum of rewards until a final time T . In an infinite horizon, a **discount factor** ($\gamma \in [1]$) is used to weight immediate rewards more heavily than distant future rewards.

Example: A beginner-friendly example is the **Cart-Pole task**. Here, the goal is to prevent a pole from falling by moving a cart left or right. The state includes the cart's position and velocity, alongside the pole's angle and angular velocity. The agent receives a reward of +1 for every time step the pole remains upright. This task forces the agent to balance immediate movements with the long-term goal of stability.

1.1.5 Policy, Value Functions and Q-functions

To evaluate how “good” a state or action is, Reinforcement Learning uses specific functions that estimate the future rewards an agent can expect. These functions serve as the bridge between simply reacting to the environment and planning for long-term success.

A **Policy** (π) is the strategy defining the probability of taking an action in a given state, essentially acting as the mapping of states to actions that the agent follows. The **Value Function** ($v_\pi(s)$) assesses the overall desirability of being in a specific condition by calculating the cumulative expected return of following that policy. Finally, the **Q-function** ($q_\pi(s, a)$) determines the value of taking a specific action in a state and then following the policy thereafter, which helps the agent distinguish which specific actions are best within a given state.

Example: Imagine a robot navigating a maze. The **policy** decides which direction to move in each position (for example, “go left” or “go forward”). The **value function** tells how good it is to stand in a certain location, meaning how close it is to reaching the goal on average. The **Q-function** tells how good a specific action is in that location, for example how good it is to move left versus right from the same position, meaning taking a specific action at this location and then keep following the policy π until the end.

1.1.6 The Recursive Nature of the Bellman Equations

The core insight allowing these functions to be calculated is the recursive nature of rewards. Because the total return at the current time is equal to the immediate reward plus the discounted return of the next time step ($G_t = R_{t+1} + \gamma G_{t+1}$), we can express the value of the current state based on the values of its successor states. The **Bellman Equation** for v_π defines the value of a state under a specific strategy. It states that the value of a state is the average over all possible actions and transitions, calculated as the immediate reward plus the discounted value of the resulting next state:

$$v_\pi(s) = \sum_a \pi(a|s) \sum_{s', r} p(s', r|s, a) [r + \gamma v_\pi(s')] \quad (2)$$

Example: To make this clearer, think of the Bellman Equation as a way to evaluate your current habits (your policy π). If your policy is to always take the bus to work, the equation calculates your total expected journey time by looking at your current bus stop (state s). It breaks the journey into two manageable parts:

1. **Immediate Reward:** The actual time spent traveling from your current stop to the very next stop.
2. **Future Value:** Your “predicted time to go” once you reach that next stop, assuming you continue to follow your habit of taking the bus for the rest of the trip.

By adding the time it took to reach the next stop to your best estimate of the remaining journey, you can constantly update your “predicted total time” without needing to wait until you actually arrive at the office. This recursive logic allows an agent to evaluate the quality of its current strategy step-by-step

1.1.7 The Bellman Optimality Equation

While the regular Bellman equation evaluates a given policy, the **Bellman Optimality Equation** seeks the absolute best performance possible. It defines the value of a state under the **Optimal Policy** (π^*), which is the strategy that maximizes the sum of rewards across all states. For the optimal state-value function (v^*), we no longer average over all actions in our policy. Instead, we look at all possible actions and choose the one that yields the maximum possible reward plus the discounted value of the best next state:

$$v^*(s) = \max_a \sum_{s', r} p(s', r|s, a) [r + \gamma v^*(s')] \quad (3)$$

Similarly, the optimality equation for the Q-function (q^*) is expressed as:

$$q^*(s, a) = \sum_{s', r} p(s', r|s, a) [r + \gamma \max_{a'} q^*(s', a')] \quad (4)$$

Example: In beginner-friendly terms, the Optimal Bellman Equation is like using a GPS to find the fastest route home. It doesn't care about your usual habits, it looks at every possible turn you could take and assumes that, for every future intersection, you will make the absolute best choice. By solving these equations, an agent can discover the optimal strategy for any given environment.

1.2 Planning and Model Assumptions

1.2.1 Dynamic Programming: Policy and Value Iteration

If the environment model is known, we can solve for the optimal policy using Dynamic Programming. **Policy Iteration** involves two alternating phases: **Evaluation** (calculating the value function for the current policy) and **Improvement** (testing different actions to see if they yield better results). Because there are a finite number of policies in a finite MDP, this process is **guaranteed to converge to the optimal policy**. An advantage of this approach is the option to parallelize the computation during the improvement stage, as the optimal action for each individual state can be determined independently using the fixed value function from the evaluation phase, allowing the updates for the entire state space to be calculated simultaneously across multiple processors (threads).

Value Iteration, instead of waiting for the evaluation to converge across all states multiple times, combines **evaluation and improvement into a single update** operation, evaluating each state only once per iteration. However, both methods become infeasible for environments with massive state spaces, such as Atari games, where every pixel combination represents a distinct state.

Policy Iteration vs. Value Iteration: Policy Iteration has higher per-iteration cost due to policy evaluation ($\mathcal{O}(S^3)$ with direct linear solvers and $\mathcal{O}(S^2A)$ for improvement), but typically converges in fewer iterations. Value Iteration has lower per-iteration complexity ($\mathcal{O}(S^2A)$ per iteration), but usually requires more iterations to converge.

1.2.2 Model-Based vs. Model-Free Methods

Up until this point, the techniques we have explored, such as Policy and Value Iteration, have operated in environments where we had complete knowledge of the state transitions. These are known as **Model-Based** methods. In this setting, the agent has access to a "model" of the environment, which includes the **transition matrix** $p(s'|s, a)$. This matrix acts like the "physics" of the world, telling the agent exactly how likely it is to move to a new state after taking a specific action. Because the agent knows these rules, it can use Dynamic Programming to plan its strategy mathematically without ever having to actually "experience" the environment. However, in many practical scenarios, the transition matrix is either unknown or far too complex to calculate. For these situations, we utilize **Model-Free** methods. Instead of trying to estimate or learn the underlying rules of the world, a model-free agent learns by interacting with the environment directly and modeling it based on its own sampling.

Example: A helpful way to visualize the difference is to think about learning how to ride a bicycle. A **model-based** approach would involve studying the physics of balance, gravity, and tire friction to calculate the exact force needed for every turn before ever getting on the bike. A **model-free** approach is the way most people actually learn: you get on the bike, fall a few times, and eventually learn which body movements keep you upright through trial and error. While model-based methods allow for perfect planning when the rules are known, model-free methods like Monte Carlo (MC) and Temporal Difference (TD) learning are essential for complex tasks where the "physics" of the environment is unavailable.

1.3 Policy & Exploration

1.3.1 Monte Carlo Sampling Methods

Monte Carlo (MC) methods are the simplest form of model-free learning. They rely on **sampling**, where the agent follows a policy to generate complete sequences of transitions, known as **episodes**. MC methods are strictly episodic, meaning they only work in environments with a clear termination point. A defining

characteristic of MC is that the agent **must wait until the end of the episode** to know the total return before it can update its knowledge, it does not "bootstrap" or guess based on other estimates. There are two primary ways to handle multiple occurrences (visits) of the same state within an episode:

- **First-visit MC:** Estimates the value of a state by averaging only the returns that follow its first occurrence within each episode. If the agent visits the same state multiple times during a single sequence, only the first visit is used to update the value approximation, while all subsequent visits in that same episode are ignored.
- **Every-visit MC:** Estimates the value of a state by averaging the returns from every visit to that state across all sampled episodes. In this variant, each time a state is encountered, even multiple times within the same trajectory, the resulting return is recorded and factored into the running average for that state.

1.3.2 On-Policy, Off-Policy, and Exploration

On-policy methods improve the policy currently used for decisions, while **Off-policy** methods learn a target policy from data generated by a different "behavior" policy. Off-policy learning is powerful because it allows an agent to learn from human experts or previous versions of itself, often using **Importance Sampling** to account for the difference in trajectory probabilities between the two policies.

A critical limitation of on-policy methods is that they will never converge to the absolute optimal policy (π^*) as long as they maintain a permanent exploration component. Because on-policy algorithms evaluate the policy they are actually following, the learned value function reflects the expected returns of the exploratory (stochastic) strategy rather than the perfect (deterministic) optimal strategy.

1.3.3 Online vs. Offline RL

The distinction between how data is collected and how it is used defines the learning setting:

1. **Online RL:** The agent learns while actively interacting with the environment. This can be **On-policy** (learning from the same policy it is using to make decisions) or **Off-policy** (learning from its own past "replay" data).
2. **Offline RL:** The agent can't interact with the environment and must learn strictly from a fixed, pre-collected dataset, such as **human expert replays**. it is **Off-policy** only.

1.3.4 Importance Sampling

In many scenarios, specifically in **Off-policy learning**, a significant mathematical challenge arises: the trajectories in the available data are different from those that would be "recommended" by the policy the agent is trying to learn. For instance, the agent might use an ϵ -greedy strategy as its **behavior policy** (b) to explore the environment, but it ultimately wants to learn a deterministic **target policy** (π) that is perfectly optimal. To calculate the expected rewards for the target policy using data generated by the behavior policy, we use **Importance Sampling**. This technique calculates the relative probability of a trajectory under the two different policies. Mathematically, it is expressed as the product of the ratios of the probabilities of taking certain actions in specific states:

$$\rho = \prod_{k=t}^{T-1} \frac{\pi(a_k|s_k)}{b(a_k|s_k)} \quad (5)$$

By weighting the observed returns (G_t) by this importance sampling ratio, the agent can accurately estimate the value of its target policy even though it is "watching" or performing a different behavior. This allows the agent to learn from diverse experiences, such as its own past exploration or even human expert replays, while still converging toward the best possible strategy.

Offline RL is inherently **Off-policy**. Because the trajectories in the data were generated by a **behavior policy** (b) that is different from the **target policy** (π) the agent is trying to learn, we use **Importance Sampling**. This mathematical technique weights the rewards based on the relative probability of an action being taken by the target policy versus the behavior policy, effectively "bridging the gap" between the two.

1.3.5 The ϵ -Greedy Algorithm

A core challenge in RL is the balance between **Exploitation** (choosing the action we think is best) and **Exploration** (trying new actions to see if they yield better rewards). A purely greedy algorithm always selects the action with the highest estimated action-value, which can lead the agent to get "stuck" on sub-optimal actions forever because it never experiments with alternatives. To resolve this, the **ϵ -greedy algorithm** ensures the agent continues to explore by introducing a random component:

- With probability $1-\epsilon$, the agent selects the greedy action (exploitation).
- With probability ϵ , the agent selects a random action (exploration).

By maintaining this ability to explore, the agent is guaranteed to eventually find the optimal policy. However, a trade-off exists: because the agent never stops choosing random actions, it will never be perfectly optimal, as it will always incur some loss from those random choices. The policy will be optimal but the agent will keep choosing a different action with a probability of ϵ .

1.3.6 Bootstrapping in Reinforcement Learning

In Reinforcement Learning, **bootstrapping** refers to updating value estimates using other current estimates rather than waiting for a final return. Monte Carlo methods do not use bootstrapping, since they learn only from complete episodes using the actual return G_t . In contrast, Temporal-Difference (TD) learning use bootstrapping by updating estimates based on the value of previous states. TD learning is considered a hybrid approach because it samples from the environment like Monte Carlo methods but bootstraps like Dynamic Programming.

The main advantage of bootstrapping is its **efficiency**, as it allows learning and updating value estimates after each step, even from incomplete episodes. This is expressed by the TD(0) update rule:

$$V(s) \leftarrow V(s) + \alpha [R + \gamma V(s') - V(s)] \quad (6)$$

Bootstrapping enables learning in **continuing (infinite-horizon) environments**, where a final return G_t may never be observed.

1.3.7 Temporal Difference (TD) Learning

TD learning is a hybrid approach that combines MC sampling with the "bootstrapping" of Dynamic Programming. Unlike MC, which requires a complete episode to finish before updating, TD updates its estimates after every single step using an estimate of the future.

Example: Consider the **transportation example**: if you are driving home and encounter a storm, your "predicted total travel time" increases immediately. You do not wait until you are home to realize the storm slowed you down, you update your expectation mid-journey based on your new "time to go" estimate.

1.3.8 SARSA & Q-Learning

Temporal Difference (TD) control methods update action-value estimates (Q -functions) after every step, combining the sampling nature of Monte Carlo with the bootstrapping of Dynamic Programming.

1. **SARSA (On-policy TD method)**: derives its name from the events used in its update: $S_t, A_t, R_{t+1}, S_{t+1}, A_{t+1}$. As an **on-policy** method, it attempts to improve the same policy that the agent is currently following to make decisions.

- **The Update Rule:** The Q -value is updated based on the action a_{t+1} actually chosen by the agent's current policy (e.g., ϵ -greedy) in the next state:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [R_{t+1} + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)] \quad (7)$$

where α is the learning rate and γ is the discount factor.

Because SARSA updates its estimates based on the actual trajectory taken, it inherently factors in the risks associated with the behavior policy's exploration.

2. **Q-Learning (Off-policy TD method):** is an **off-policy** method, meaning it estimates the value of the **optimal policy** regardless of the "behavior policy" the agent is using to explore the environment.

- **The Update Rule:** Instead of looking at the actual next action taken, Q-Learning updates the current Q -value using the **maximum possible** Q -value in the successor state:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[R_{t+1} + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)] \quad (8)$$

This update assumes the agent will follow a greedy approximation of the optimal policy in the next step, even if the current behavior policy actually chooses a random action for exploration.

Example: The **Cliff Walk example** illustrates the practical difference between on-policy and off-policy temporal-difference methods. In this grid-world task, the agent must reach a goal while avoiding a cliff that yields a reward of -100 and resets the agent to the start. **Q-learning**, as an off-policy method, learns the mathematically optimal policy π^* , which corresponds to the shortest path along the edge of the cliff. However, because learning is performed under an ϵ -greedy behavior policy, random exploratory actions are still taken. When operating close to the cliff, these random actions frequently cause the agent to fall, incurring large negative rewards. **Sarsa**, being an on-policy algorithm, evaluates and improves the policy it actually executes, including its exploratory behavior. As a result, it learns that states near the cliff are risky due to the chance of random actions and therefore converges to a safer, longer path farther from the edge.

Although Q-learning converges to a shorter path, **Sarsa achieves higher cumulative reward during training** in this scenario by avoiding repeated catastrophic penalties caused by exploration.

2 Lecture 2: Deep Q-Learning, Policy Gradients, and Actor-Critic Methods

2.1 Deep Q-Learning (DQN)

In complex environments where state spaces are too large for tables, we use **function approximation**. A Deep Neural Network (DNN) with parameters θ is used to approximate the Q -values: $q(s, a, \theta) \approx q(s, a)$. The network is trained using gradient descent to minimize a loss function derived from the Bellman optimality equation. In Atari games for example, the input is typically a stack of the last four frames, and the output layer provides Q -values for every possible action, enabling efficient decision-making. The transition from tabular Q-learning to Deep Q-learning involves using a neural network to approximate the Q -function, denoted as $Q(s, a, \theta)$, where θ represents the network parameters. The standard update rule for Q-learning is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)] \quad (9)$$

In DQN, we optimize the network by calculating the Mean Square Error (MSE) loss between the predicted value and a target:

$$Loss = \left(r + \gamma \max_{a'} Q(s', a', \theta^-) - Q(s, a, \theta) \right)^2 \quad (10)$$

2.1.1 Challenges in Basic Deep Q-Learning

While Deep Q-Networks (DQN) leverage neural networks to model the action-value function $Q(s, a, \theta)$, the basic algorithm often struggles with convergence due to the following problems:

- **Sequential Prediction and Correlated States:** In reinforcement learning, data is collected sequentially, meaning that state s_t and its successor s_{t+1} are highly correlated. Training a neural network on these non-independent, temporally related samples leads to significant instability in gradient updates.

- **Catastrophic Forgetting:** Because the agent trains on data as it is experienced, it tends to overfit to recent information and “forget” older experiences. For example, a network may learn to navigate Level 2 of a game but, in doing so, override the parameters that allowed it to succeed in Level 1.
- **Non-Stationary Targets:** The DQN update relies on a target derived from the Bellman equation:

$$r + \gamma \max_{a'} Q(s', a', \theta).$$

Because the **same parameters** (θ) are used for both the current value estimation and the Q-target, any update to θ moves both quantities simultaneously. This creates a feedback loop often described as “**chasing a moving target**”, which prevents the model from reaching a stable equilibrium.

2.1.2 Fixed Q-Targets

To improve stability, the **Fixed Q-Targets** method “freezes” the target parameters θ^- for a fixed number of iterations (T steps). This prevents the target from moving constantly during every update, providing a more stable objective for the network to converge toward.

2.1.3 Experience Replay

Training on sequential trajectories leads to high correlation between states and a risk of the network “forgetting” previous experiences as it overfits to recent data. **Experience Replay** addresses this by storing trajectories in a **replay buffer** and sampling them randomly for training.

- **Random Sampling:** Simple and effective for breaking correlation.
- **Prioritized Experience Replay (PER):** Instead of uniform sampling, this approach prioritizes transitions with a high **TD error** (δ), as these samples likely contain the most useful information for the model to learn.
- **Stochastic Prioritization:** Stochastic prioritization balances between greedy and uniform sampling to avoid overfitting to noisy transitions and neglecting parts of the replay buffer. Each transition i is sampled with probability:

$$P(i) = \frac{p_i^\alpha}{\sum_k p_k^\alpha} \quad (11)$$

Where transitions are ranked by TD error magnitude. The hyperparameter α controls prioritization strength:

- $\alpha = 0$: uniform sampling
- $\alpha = 1$: fully greedy prioritization

2.2 Value-Based Methods (Advanced DQN Architectures)

2.2.1 The Advantage Function

The **advantage function**, $A(s, a)$, quantifies the relative importance of taking a specific action compared to the average value of being in a given state. It is formally defined through its relationship with the state-value and action-value functions:

$$Q(s, a) = V(s) + A(s, a) \quad (12)$$

State vs. Action Importance: In some states, the choice of action is largely irrelevant (e.g., when already falling off a cliff), while in other states the specific action choice is crucial (e.g., evading an obstacle).

2.2.2 Double DQNs

A major problem in Q-learning is the production of over-optimistic Q-values caused by using the same estimator for both action selection and evaluation. **Double Q-learning** addresses the challenge of **inaccurate, over-optimistic Q-values** by utilizing two separate estimators, Q^A and Q^B . Each function acts as an unbiased estimator for the other: one determines the best action while the other evaluates its value. When updating Q^A , the algorithm selects the optimal action a^* using Q^A but computes the update using the value estimated by Q^B :

$$Q^A(s, a) \leftarrow Q^A(s, a) + \alpha \left(r + \gamma Q^B \left(s', \arg \max_a Q^A(s', a) \right) - Q^A(s, a) \right) \quad (13)$$

In Deep Q-Networks (DDQN), this idea is implemented by decoupling action selection from action evaluation. The **online network** with parameters θ is used to select the greedy action, while the **target network** with parameters θ^- is used to evaluate its value. This prevents the maximum operator from acting on noisy estimates and reduces overestimation bias.

2.2.3 Dueling DQNs

The **Dueling DQN** architecture improves learning by recognizing that the importance of choosing a specific action is not equal across all states. To capture this intuition, the architecture **decouples** the representation by splitting into two separate streams:

1. **Value Stream:** Estimates a scalar state-value $V(s)$, representing how “good” it is to be in that state regardless of the action taken.
2. **Advantage Stream:** Estimates a vector $A(s, a)$, representing the “advantage” or relative importance of each action in that specific state.

Combining the Streams: Although one might simply add the two outputs ($Q = V + A$), that approach does not allow the model to clearly differentiate between the individual contributions of the state and the action. To ensure the network can distinct these contributions, the streams are combined using the following equation:

$$Q(s, a, \theta, \alpha, \beta) = V(s; \theta, \beta) + \left(A(s, a; \theta, \alpha) - \frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha) \right) \quad (14)$$

Where as $V(s; \theta, \beta)$ is the state-value function which is a scalar output, $A(s, a; \theta, \alpha)$ is the advantage function which is a vector output for each action, (θ, α, β) are the parameters of the neural network, $|\mathcal{A}|$ is the size of the action space, and $\frac{1}{|\mathcal{A}|} \sum_{a'} A(s, a'; \theta, \alpha)$ is the mean advantage across all possible actions. subtracting this term stabilizes the identity of $V(s)$ and $A(s, a)$.

This architectural shift allows the agent to identify valuable or dangerous states independently of the actions, such as recognizing a “no-win” situation where the state-value is low regardless of the agent’s choice.

Example: For instance, if a car is already falling off a cliff, it no longer matters how the driver turns the wheel, as the state itself is inherently low-value regardless of the action taken. Conversely, when an obstacle is approaching, the specific action of “evading” becomes paramount to success.

2.3 Policy Gradient Methods

Unlike value-based methods that rely on estimating state or action values, **Policy Gradient** methods directly model the policy $\pi(a | s, \theta)$ using parameterized approximations such as neural networks. These methods seek to improve performance by performing gradient ascent on a performance measure $J(\theta)$, updating parameters as:

$$\theta_{t+1} = \theta_t + \alpha \nabla J(\theta).$$

Key advantages include the ability to learn stochastic policies, natural handling of continuous action spaces, and smoother changes in action probabilities compared to ϵ -greedy methods.

2.3.1 REINFORCE Algorithm

REINFORCE is the standard **Monte Carlo policy gradient** algorithm. It estimate the gradient of performance based on sampled trajectories. Parameters are updated at the end of each episode based on the return G_t received from step t :

$$\theta_{t+1} = \theta_t + \alpha \gamma^t G_t \nabla \ln \pi(A_t | S_t, \theta_t) \quad (15)$$

The update is proportional to the reward received and in **reverse proportion** to the probability of the action taken. Thus, rarer actions that result in high rewards receive larger probability updates. Because it relies on full-trajectory returns, REINFORCE is applicable only to episodic settings and often suffers from high variance and slow convergence.

2.3.2 REINFORCE with Baseline

Because it relies on sampling full trajectories, the basic REINFORCE algorithm often suffers from high variance, leading to fluctuations and slow convergence. To reduce large and unstable updates, a baseline $b(s)$ is introduced as a reference value. **It does not change the learning direction (does not introduce bias)**, but it scales down the size of the updates, leading to more stable learning. **The baseline must not depend on the chosen action a .** If it did, it would interfere with the learning objective and introduce bias into the policy update.

The baseline is subtracted from the return G_t so that the update depends on how much better or worse an action performed compared to the average for that state. Actions with above-average outcomes are reinforced, while actions with below-average outcomes are discouraged. A common and effective choice for $b(s)$ is the estimated state-value function $v(S_t, w)$ as it's not depend on a specific chosen action a , where w denotes the parameters (weights) of the value function $v(s, w)$. It represents the learned model that estimates how good a state s is. The difference between the observed return and the baseline defines the advantage signal:

$$\delta \leftarrow G_t - v(S_t, w) \quad (16)$$

This signal rewards actions that outperform the expected value of the state. At each time step t in an episode, the value-function parameters w and the policy parameters θ are updated simultaneously:

$$w \leftarrow w + \alpha_w \gamma^t \delta \nabla v(S_t, w) \quad (17)$$

$$\theta \leftarrow \theta + \alpha_\theta \gamma^t \delta \nabla \ln \pi(A_t | S_t, \theta) \quad (18)$$

So at the end we're updating based on the unbiased δ rather than the noisy G_t .

2.3.3 Proximal Policy Optimization (PPO)

Proximal Policy Optimization (PPO) is a state-of-the-art algorithm designed to ensure stable and efficient training by constraining how much a policy can change in a single update. It prevents “destructive” large updates that could catastrophically degrade performance by ensuring the new policy remains proximal (close) to the old one.

- **Probability Ratio:** PPO tracks the ratio $r_t(\theta)$ between the new policy and the old policy to measure the magnitude of the change:

$$r_t(\theta) = \frac{\pi_\theta(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} \quad (19)$$

- **Clipped Surrogate Objective:** To maintain stability, the objective restricts $r_t(\theta)$ to a safe range $[1 - \epsilon, 1 + \epsilon]$ (typically $\epsilon \in [0.1, 0.2]$). This prevents the update from moving the policy too far based on a single sample:

$$L^{\text{CLIP}}(\theta) = \hat{\mathbb{E}}_t \left[\min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right] \quad (20)$$

The **minimum operation** ensures the algorithm chooses the more conservative estimate, effectively “braking” the update if it becomes too aggressive.

- **Entropy Bonus:** The full PPO objective includes an entropy bonus $S[\pi](s_t)$.

Entropy: In reinforcement learning, entropy is a measure of the **uncertainty or randomness** in the policy’s action distribution. A high entropy value means the agent’s actions are more spread out (encouraging **exploration**), while a low entropy value means the policy is becoming deterministic (encouraging **exploitation**). The bonus rewards the agent for maintaining uncertainty, preventing the policy from “collapsing” into a single action too early in training.

2.4 Actor-Critic Methods

Actor-Critic methods are hybrid architectures designed to combine the advantages of policy-based methods (which model the policy directly) and value-based methods (which provide stable value estimates). This combination reduces the high variance often seen in pure policy gradients while maintaining the ability to learn stochastic policies and handle continuous action spaces.

- **The Actor:** Represents the parameterized policy $\pi(a | s, \theta)$. Its role is to select actions. It is updated using gradients to increase the probability of actions that lead to “better-than-expected” outcomes.
- **The Critic:** Represents the state-value function $v(s, w)$. Its role is to evaluate the current state. It provides a baseline (a “critique”) that informs the actor whether the action taken was better or worse than the average expectation for that state.

2.4.1 One-step Actor-Critic and the TD Error

In the **One-step Actor-Critic** framework, the agent utilizes **bootstrapping** to update its parameters after every single step rather than waiting for the end of an episode. The feedback is provided by the **Temporal Difference (TD) error** δ , which serves as the advantage signal for both components:

$$\delta \leftarrow R + \gamma v(S', w) - v(S, w) \quad (21)$$

- **Actor Update:** If $\delta > 0$, the action performed better than the Critic’s average estimate, and the Actor is updated to increase the probability of that action.
- **Critic Update:** The Critic updates its parameters w to reduce the error δ , making its future estimate of $v(S, w)$ more accurate.

2.4.2 Asynchronous Advantage Actor-Critic (A3C)

A3C is a sophisticated architecture that improves training efficiency and stability through parallelization:

- **Parallel Workers:** Multiple independent agents (workers) interact with separate environments simultaneously. Each worker calculates its own gradients based on its unique experience.
- **Global Network:** Workers asynchronously update a single shared **global network**. This master network maintains the authoritative policy and value function.
- **Breaking Correlation:** By running agents in parallel, the algorithm breaks the temporal correlation of data. This diversity of experience prevents the model from getting stuck in local optima and improves sample efficiency.

3 Lecture 3: Imitation Learning

This lecture focuses on **Imitation Learning (IL)**, where the goal is to learn a policy by mimicking an expert, and the theoretical framework of **regret** used to evaluate reinforcement learning algorithms.

3.1 Regret in RL Algorithms

Regret is defined as the difference in reward between following an optimal policy π^* and a current policy π . The action-value is the expected reward for an action:

$$Q(a) = \mathbb{E}[r \mid a].$$

The optimal value is the maximum possible action-value:

$$V^* = \max_a Q(a).$$

Single-step regret (ℓ_t): The loss of reward for not following π^* for one step:

$$\ell_t = \mathbb{E}[V^* - Q(a_t)].$$

Total regret (L_T): The cumulative loss over T steps:

$$L_T = \mathbb{E} \left[\sum_{t=1}^T (V^* - Q(a_t)) \right].$$

Example: Greedy and ϵ -greedy methods can perform poorly over time because they can keep exploring forever, while decaying ϵ -greedy improves performance by slowly reducing exploration as learning progresses.

3.2 Introduction to Imitation Learning

Imitation Learning aims to create policies that maximize rewards by mimicking an expert policy π^* , rather than learning accurate value functions.

- **Challenges:** Reinforcement Learning is difficult because states are not independent and identically distributed (i.i.d.) since each action affects the next state. Instead, current states depend on previous actions, creating a sequential prediction problem. Small behavioral changes can lead to large reward changes due to non-linear dynamics.
- **The Compounding Error Problem:** A small mistake can move the learner into unfamiliar states, causing errors to grow over time and making recovery difficult.

3.3 Imitation Learning Algorithms

3.3.1 Apprenticeship Learning

Apprenticeship Learning is considered the simplest and most foundational form of imitation learning. In this framework, the agent acts as a student by observing an expert (π^*) and attempting to reconstruct the logic of the task through the following iterative process:

1. **Observation:** The agent watches the expert perform a task and records the resulting state-action trajectories.
2. **Model Learning:** These trajectories are used to learn a **dynamics model**, specifically the environment's state-action transition matrix.
3. **Policy Optimization:** The agent applies RL methods to this learned model to derive a near-optimal policy.
4. **Refinement:** If the resulting policy's utility is insufficient, the agent collects new trajectories and repeats the learning process.

Apprenticeship Learning is **completely greedy**, relying solely on exploitation with no active exploration of the state space. As a result, it **lacks robustness** and cannot handle with previously unexplored states or recover from errors: if the agent deviates from the expert’s demonstrated path, it enters states for which it has neither data or a strategy. Moreover, because the environment dynamics and the policy are learned jointly, the resulting model often suffers from **poor generalization** to new or slightly altered states.

Example: The learner acts like a student copying a master. It takes notes on every move the master makes to build a “map” (the dynamics model) of how the world works. Once it thinks it understands the map, it uses RL to find the best path through it. If it fails when testing the path, it goes back to watch the master again to fill in the gaps in its map.

3.3.2 Supervised Learning (Behavioral Cloning)

This “naive” approach reduces a sequential imitation problem into a series of decoupled supervised learning tasks. It employs a **stationary policy**, meaning the decision-making rule remains constant throughout the entire trajectory. The algorithm seeks a policy π that minimizes the observed surrogate loss ℓ (the regret relative to the expert) under the induced state distribution:

$$\pi = \arg \min_{\pi} \mathbb{E}_{s \sim d_{\pi}} [\ell(s, \pi)] \quad (22)$$

A major challenge is that the state distribution d_{π} depends on the policy π itself, creating a non-i.i.d. supervised learning problem in which actions influence future inputs.

Example: The learner tries to memorize the expert’s actions like a set of “flashcards.” However, it only studies the specific situations the expert encountered. If the learner makes one wrong move, it ends up in a “new” situation it never studied and does not know how to get back on track. The biggest issue with this approach is its inability to recover from failure. Once the agent makes a mistake, it finds itself in an unseen state where subsequent actions are also likely to be mistakes. This causes errors to compound, leading to super-linear growth in the total cost $J(\pi)$.

3.3.3 Forward Training

Forward Training is an imitation learning algorithm designed to address the **compounding error** problem by training a **non-stationary policy** (a sequence of different policies $\{\pi_1, \pi_2, \dots, \pi_T\}$), where a specific policy is learned for each time step t in the task horizon. At each iteration i , it samples T -step trajectories by following the policies learned in previous iterations $(\pi_1, \dots, \pi_{i-1})$. It then queries the expert for the optimal action $\pi^*(s_i)$ at that specific time step i to train a new classifier, π_i . This process repeats until a policy is trained for every step in the horizon T .

Example: Instead of trying to learn one “master” rule for the whole journey, the learner is taught step-by-step. The expert watches the learner perform the first $i - 1$ steps and then provides the “correct answer” only for the i -th step. This allows the learner to see the actual states it ends up in and learn how to move forward from there. The total regret for this algorithm is bounded by $O(uT\epsilon)$, which is **near-linear**. This is a significant improvement over standard behavioral cloning.

Pros and Cons:

- **Pros:** Since the algorithm is trained on states it actually visits, it makes better decisions in those states and can recover from mistakes using expert feedback.
- **Cons:** It is computationally expensive because it requires iterating over all T steps individually. This makes it impractical for real-world applications with very large or infinite horizons.

3.3.4 DAgger (Dataset Aggregation)

DAgger (Dataset Aggregation) is an imitation learning algorithm that uses a **stationary policy**, meaning the agent follows the same set of rules at every step. It is specifically designed to solve the “chicken and

egg” problem: to train a model effectively, we need data from the states it will encounter, but those states cannot be known until the model begins taking actions. To address this challenge, DAgger uses a simple, repeating process:

1. **Initialize:** Start with an initial dataset D (often from expert demonstrations) and an initial policy.
2. **Sample:** Run the current policy π_i to collect a new set of trajectories D' .
3. **Label:** Query the expert to provide the optimal actions $\pi^*(s)$ for every state encountered in D' .
4. **Aggregate:** Update the main dataset by combining the old and new data: $D \leftarrow D \cup D'$.
5. **Train:** Train a new policy π_{i+1} on the expanded dataset and repeat.

Example: The learner takes a “practice run” alone. Afterward, it shows the expert every state it visited and asks, “What should I have done here?” It adds these new “corrected” examples to its study guide and tries again. DAgger is more efficient than forward training because it updates the policy only once per trajectory and uses data from the entire trajectory. It achieves **near-linear regret**.

3.3.5 DAgger with Coaching

Standard DAgger can fail if the learner’s policy is too restricted to mimic the expert, much like a beginner cannot learn to be a master just by watching one. DAgger with Coaching addresses this by presenting the learner with “challenging yet feasible” examples.

The Hope Action: Instead of forcing the learner to match the expert exactly, the coach provides a hope action $\tilde{\pi}_i(s)$ that balances being optimal with being achievable for the current policy:

$$\tilde{\pi}_i(s) = \arg \max_{a \in \mathcal{A}} (\lambda_i \cdot \text{score}_{\pi_i}(s, a) - L(s, a)) \quad (23)$$

Example: Instead of the expert giving the “perfect” answer that might be too hard to execute, a coach gives a “better” answer that is still within the learner’s current reach. As the learner improves, the coach gradually increases the difficulty.

4 Lecture 4: Multi-Arm Bandits

This lecture explores the fundamental trade-off between **exploration** and **exploitation** in the simplified framework of Multi-Arm Bandits (MAB). Unlike the full reinforcement learning problem, MAB involves learning to act in only one situation (one state), allowing us to clearly see how feedback mechanisms work.

4.1 The Multi-Arm Bandit (MAB) Problem

The MAB problem is modeled as a player facing a row of K slot machines (“arms”) with unknown distributions.

- **The Process:** At each time step t , the agent selects one arm and receives a reward. The agent only observes the reward of the selected arm and remains ignorant of the others.
- **Exploration vs. Exploitation:** The player must balance exploration (discovering new information) with exploitation (using known information to maximize immediate rewards).
- **Objective:** Minimize the **expected cumulative regret**, defined as the difference between the reward of the optimal strategy and the actual strategy:

$$\mathbb{E}[\text{Reg}_n] = n \cdot R^* - \sum_{i=1}^n \mathbb{E}[r_i] \quad (24)$$

4.2 Simple Bandit Algorithms (Context-Free)

4.2.1 ϵ -greedy Algorithm

This algorithm exploits the current best-known option most of the time (with probability $1 - \epsilon$) but explores a random arm with probability ϵ .

- **Stationary Problem:** In environments where reward distributions remain fixed over time, the goal is to converge to the true mean reward of each arm. This is achieved using a **sample-average update** rule:

$$Q_{n+1} = Q_n + \frac{1}{N(A)} (R_n - Q_n) \quad (25)$$

In this setting, the step size $1/N(A)$ decreases as the number of trials $N(A)$ increases, ensuring that the estimate becomes more stable as more data is collected.

- **Non-stationary Problem:** In environments where rewards change over time, the agent must be able to “forget” old data and prioritize recent information. This is achieved using a **constant step size** $\alpha \in (0, 1]$:

$$Q_{n+1} = Q_n + \alpha (R_n - Q_n) \quad (26)$$

This update rule results in an **exponentially decaying weighted average**, where the influence of older rewards decreases over time:

$$Q_{n+1} = (1 - \alpha)^n Q_1 + \sum_{i=1}^n \alpha (1 - \alpha)^{n-i} R_i \quad (27)$$

This allows the agent to track shifting reward distributions rather than converging to a static long-term average.

ϵ -greedy explores **indiscriminately**, it does not account for how close a non-greedy arm is to being optimal or for the uncertainty in its reward estimate.

4.2.2 Upper Confidence Bound (UCB)

The Upper Confidence Bound (UCB) algorithm improves upon ϵ -greedy by addressing its primary weakness: indiscriminate exploration. While ϵ -greedy explores by picking any non-greedy action with equal probability, UCB selects actions based on their potential to be optimal, considering both their current value estimates and the uncertainty associated with those estimates.

$$A_t = \arg \max_a \left[Q_t(a) + c \sqrt{\frac{\ln t}{N_t(a)}} \right] \quad (28)$$

- **Value Term** ($Q_t(a)$): Represents the current estimate of the action’s reward based on historical data. It favors exploitation by prioritizing actions that have performed well in the past.
- **Exploration / Uncertainty Term** ($c \sqrt{\frac{\ln t}{N_t(a)}}$): Provides an **exploration bonus** that accounts for uncertainty in the estimate.
 - The term $\ln t$ ensures the bonus grows over time as the agent gains more overall experience.
 - The term $N_t(a)$ in the denominator ensures the bonus shrinks for actions that have been selected frequently, reflecting increased confidence in their estimated value.
 - The hyperparameter c controls the overall magnitude (or “aggressiveness”) of exploration.

UCB is a **deterministic** action-selection method, meaning that for a given set of value estimates and selection counts, it will always choose the same arm. It effectively bounds the expected number of pulls of sub-optimal arms, often leading to higher average rewards over time compared to ϵ -greedy. However, UCB performs poorly in **non-stationary** environments where reward distributions change over time.

4.3 Advanced Bandit Algorithms

The following algorithms extend the basic bandit framework by either learning relative action preferences, external state information (context), or Bayesian uncertainty to guide exploration.

4.3.1 Gradient Bandit Algorithms

Unlike methods that estimate absolute action values, Gradient Bandit Algorithms learn a **numerical preference** $H_t(a)$ for each action. These preferences do not represent predicted rewards, instead, only the relative differences between preferences determine the probability of selecting an action.

- **Softmax Policy:** The probability of choosing action a follows a softmax distribution:

$$\pi_t(a) = \frac{e^{H_t(a)}}{\sum_{b=1}^K e^{H_t(b)}} \quad (29)$$

- **Baseline Subtraction:** To improve convergence and reduce the variance of the gradient estimates, the algorithm subtracts a baseline—the average reward observed so far, denoted by $\bar{R}$.
- **Update Rule:** The preference is updated based on the difference between the observed reward R_t and the baseline:

$$H_{t+1}(a) = H_t(a) + \alpha(R_t - \bar{R})(\mathbb{I}_{a=A_t} - \pi_t(a)) \quad (30)$$

If the reward is higher than average, the preference for the selected action increases, while the preferences for all other actions decrease proportionally.

4.3.2 Contextual Bandits

Standard bandits do not consider the state (context) in which actions are chosen. Contextual Bandits bridge the gap to full reinforcement learning by using feature vectors $x_{t,a}$ (e.g., user attributes or item categories) to inform decisions. The expected reward for arm a is modeled as a linear function of its context:

$$\mathbb{E}[r_{t,a} \mid x_{t,a}] = x_{t,a}^\top \theta_a^* \quad (31)$$

where θ_a^* is an unknown parameter vector.

LinUCB Algorithm: The parameters are estimated using regression:

$$\hat{\theta}_a = (D_a^\top D_a + I_d)^{-1} D_a^\top b_a \quad (32)$$

The selected arm maximizes the estimated reward plus a confidence interval:

$$a_t = \arg \max_{a \in \mathcal{A}_t} \left(\hat{\theta}_a^\top x_{t,a} + \alpha \sqrt{x_{t,a}^\top \mathbf{A}_a^{-1} x_{t,a}} \right) \quad (33)$$

This criterion balances exploitation (choosing arms with high predicted reward) and exploration (favoring arms with greater uncertainty), ensuring that new or uncertain arms receive sufficient opportunities to be selected.

Example: The LinUCB algorithm is particularly powerful in real-world applications such as news recommendation systems because it **efficiently handles changing environments**, where the set of available actions (arms) is not fixed. In many applications, the set of available arms changes over time. For example, in news recommendation, new articles are continuously added while older ones are removed. Standard bandit algorithms often struggle with new arms because they rely on historical data tied to a specific arm. LinUCB instead models the expected reward as a linear function of contextual features $x_{t,a}$, allowing it to generalize across arms. When a new article appears, LinUCB uses its context vector (e.g., category, length, or keywords) together with the learned coefficient vector θ_a^* to estimate its expected reward. This enables the algorithm to make informed decisions immediately, even for arms with no prior interaction history.

4.3.3 Thompson Sampling

Thompson Sampling is a Bayesian heuristic that provides a natural solution to the exploration–exploitation problem. Instead of estimating fixed action values, it maintains a belief distribution (posterior) over the unknown parameters of each arm and selects actions according to the probability that they are optimal. At each time step t , the algorithm proceeds as follows:

1. **Sample:** Sample a random variable X_a from each arm’s current belief distribution.
2. **Act:** Select and play the arm i with the largest sampled value X_i .
3. **Observe:** Receive the reward r from the environment.
4. **Update:** Update the belief distribution of the selected arm based on the observed reward.

For arms with Bernoulli rewards (such as click or no-click), the **Beta distribution** is used as a **conjugate prior**. This means that the agent’s belief about an arm can be summarized using only two values: the number of observed successes and failures. When a new outcome is observed, the update is extremely simple: a success increases the success count by one, and a failure increases the failure count by one. Because the belief always remains a Beta distribution after each update, no complex recomputation is required. This results in fast and computationally efficient belief updates. the steps are as follows:

- **Initial State:** At the beginning, when the agent has no prior knowledge, each arm starts with a **uniform prior** $\text{Beta}(1, 1)$. This represents a flat belief in which all reward probabilities are equally likely.
- **Interpretability (The Scorecard):** The parameters α and β act as a simple **scorecard** for the agent’s experience, known as pseudo-counts:
 - If the outcome is a **success** ($r = 1$), update $\alpha \leftarrow \alpha + 1$.
 - If the outcome is a **failure** ($r = 0$), update $\beta \leftarrow \beta + 1$.
- **Automatic Balancing:** The average reward of an arm is estimated as $\frac{\alpha}{\alpha + \beta}$. As more data is collected, both α and β grow, causing the distribution to become more concentrated around this average. As a result, the agent naturally reduces exploration and increasingly exploits the best-performing arms.

5 Lecture 5: Monte Carlo Tree Search, AlphaGo, and AutoML

This lecture explores Monte Carlo Tree Search (MCTS), its role in the success of AlphaGo and AlphaGo Zero, and how these reinforcement learning frameworks are adapted to solve Automated Machine Learning (AutoML) problems.

5.1 Monte Carlo Tree Search (MCTS)

Monte Carlo Tree Search (MCTS) is an algorithm that combines Monte Carlo sampling with tree search to represent states and actions in a sequential decision process. It is particularly effective because it does not require domain-specific heuristics, can be interrupted at any time to return the current best action, and operates within a fixed computational budget.

5.1.1 The steps of MCTS

MCTS iteratively executes four main steps:

1. **Selection:** Starting from the root, the algorithm navigates the tree using a **tree policy** π until a leaf node (a place in the tree where you cannot go any further yet) is reached. This policy commonly uses an Upper Confidence Bound (UCB) rule:

$$\pi_{\text{UCB}}(s) = \arg \max_a \left(Q(s, a) + c \sqrt{\frac{\ln n(s)}{n(s, a)}} \right) \quad (34)$$

2. **Expansion:** If the leaf node is not terminal, one or more child nodes corresponding to legal actions are added to the tree.
3. **Simulation (Rollout):** From the newly expanded node, a game is simulated until termination, typically using random actions or a simple rollout policy.
4. **Backpropagation:** The outcome of the simulation (e.g., win or loss) is propagated back up the tree, updating the action-value estimates $Q(s, a)$ along the visited path.

5.1.2 Combining MCTS with a Policy

The basic Monte Carlo Tree Search (MCTS) algorithm can be improved by combining a policy to guide both search and pruning. For example, the Go software *Crazy Stone* uses a policy defined by hand-crafted features to evaluate nodes after a sufficient number of simulations have been performed. If a move is identified as low-priority by the policy, it can be pruned from the search tree. This strategy slows the overall growth of the tree while focusing computational resources on moves that are more likely to be optimal.

5.1.3 Combining MCTS with a Value Function

Another improvement combines MCTS with a value function to replace or guide the random sampling process. The action-value function $Q(s, a)$ is approximated using a set of features $\phi(s, a)$ and weights θ , followed by a mapping function σ that estimates the probability of winning from a given state-action pair. Instead of relying on a purely random rollout policy, simulations can be performed using more informed strategies:

- **ϵ -greedy policy:** Selects the greedy action with high probability while choosing a random action with probability ϵ to encourage exploration.
- **Greedy policy with a noisy value function:** Chooses the action that maximizes $Q(s, a)$ while adding noise η to the value estimates in order to promote exploration.
- **Softmax distribution:** Selects actions according to a probability distribution derived from $Q(s, a)$, controlled by a temperature parameter τ that balances exploration and exploitation.

5.2 Reinforcement Learning in the Game of Go

Go is a 2,500-year-old game of perfect information originating in China. It is played on a 19×19 grid where players attempt to surround more territory than their opponent. Despite its simple rules, the game presents an astronomical state space. While chess has a branching factor of approximately $b \approx 35$ and a depth of $d \approx 80$, Go has $b \approx 250$ and $d \approx 150$. This results in a search tree complexity of b^d , making the search computationally infeasible.

5.2.1 AlphaGo Architecture

AlphaGo achieved superhuman performance by combining Monte Carlo Tree Search (MCTS) with deep neural networks and a specialized rollout policy.

- **Supervised Learning (SL) Policy Network (p_σ):** Trained on 29.4 million positions from 160,000 human games to predict strong moves by maximizing the likelihood of selecting the human action. Training took approximately three weeks on 50 GPUs.

- **Reinforcement Learning (RL) Policy Network (p_ρ):** Initialized from the SL network and further improved through self-play using the REINFORCE algorithm. The network played against previous versions of itself to discover stronger strategies.
- **Value Network (v_θ):** Trained to estimate the probability of winning from a given board state. It was trained on 50 million mini-batches and outputs a single scalar value v , significantly reducing variance during MCTS evaluations.
- **Rollout Policy (p_π):** A **linear softmax** model used during the simulation phase of MCTS to play out games to termination efficiently. A rollout is a fast, simulated game played from a given state until termination using a known policy. Its purpose is to quickly estimate whether the current state is good or bad.

AlphaGo first learned by watching millions of moves made by human experts. It then practiced by playing millions of games against itself. Finally, it used a judge (the value network) to estimate which board positions were most likely to lead to a win.

AlphaGo Stages:

1. **Supervised Policy Learning:** A policy network is trained on human expert games to predict strong moves.
2. **Reinforcement Policy Learning:** The policy is improved through self-play using the REINFORCE algorithm.
3. **Value Learning:** A value network is trained to predict the game outcome from a board position.
4. **MCTS with Rollouts:** During play, MCTS is used for search. Leaf nodes are evaluated using a combination of value network predictions and fast rollout simulations.

5.2.2 AlphaGo Zero

AlphaGo Zero represents the next generation of the architecture and learns entirely through self-play, **without using human data or hand-crafted features**. The input were shrink to a $19 \times 19 \times 17$ tensor, encoding the current board, the previous seven board states for both players, and an indicator of the current player turn.

- **Pure Self-Play:** AlphaGo Zero learned entirely through self-play, starting from random play and using only the game rules. MCTS was used to evaluate and improve the policy, and a new network replaced the previous one only after achieving at least a 55% win rate in self-play matches.
- **Single Residual Network (f_θ):** A single residual network replaces all previous networks. Given a board state s , it outputs both a policy vector P and a value estimate v .
- **No Rollouts:** AlphaGo Zero eliminates rollout simulations. Instead, leaf nodes in MCTS are evaluated directly using the value head of the neural network.

AlphaGo Zero learned to play Go by playing against itself. It uses one neural network to choose moves and estimate who is winning, without learning from human games or using fast game simulations. By repeatedly improving through self-play, it became stronger than any human player.

AlphaGo Zero Stages:

1. **Self-Play with MCTS:** The current network plays games against itself. MCTS generates training data consisting of states s , search probabilities π , and final outcomes z .
2. **Network Training:** A single neural network is trained to output both a policy P and a value estimate v from the self-play data.
3. **Evaluation:** The new network replaces the old one only if it achieves a sufficient win rate in head-to-head matches.

5.3 Applications in AutoML

Automated Machine Learning (AutoML) aims to remove the human “trial-and-error” process from the machine learning workflow. By adapting the AlphaZero framework, reinforcement learning agents can treat the process of building high-performance pipelines and neural network architectures as a strategic game.

5.3.1 AlphaD3M

AlphaD3M is an AutoML system that directly adapts the AlphaZero framework to the problem of automated machine learning pipeline generation. Instead of playing a board game, the agent “plays” the process of constructing a complete data pipeline. The entire pipeline is represented as a single state. This state includes:

- Metadata about the dataset, denoted by $f(D_i)$,
- The machine learning task T_j (e.g., classification or regression),
- The current pipeline configuration S_t .

This representation allows the agent to reason about the pipeline as a complete object rather than as independent steps. The agent acts as a single player and performs moves that modify the pipeline. Possible actions include inserting, deleting, or replacing pipeline primitives such as preprocessing steps, feature transformations, or learning algorithms. The reward signal corresponds to pipeline performance, typically measured using metrics such as accuracy or F1 score on a validation set. A higher-performing pipeline corresponds to a “win” in the AlphaZero-style search.

AlphaD3M looks at the entire pipeline at once and decides whether to change big components. It is like redesigning a whole machine rather than adjusting one small part at a time.

5.3.2 Neural Architecture Search (NAS)

Neural Architecture Search replaces manual neural network design with a learning-based controller that automatically explores and optimizes network architectures based on validation performance. An RNN generates a sequence of architectural decisions in an autoregressive manner, each sampled architecture is trained and evaluated, and its validation accuracy R is used as a reward signal. Since the reward is not differentiable, the controller is trained using the REINFORCE algorithm to maximize the expected reward $J(\theta_c) = \mathbb{E}[R]$.

Update Rule:

$$\nabla_{\theta_c} J(\theta_c) = \frac{1}{m} \sum_{k=1}^m \sum_{t=1}^T \nabla_{\theta_c} \log P(a_t | a_{1:t-1}; \theta_c) R_k \quad (35)$$

Example: Think of NAS as an automated NN designer. The controller acts like an architect who writes a blueprint step by step: first choosing the type of layer, then its size, then how it connects to other layers. Each completed blueprint is handed to a builder who trains the network and gives it a score based on accuracy. If the score is high, the architect learns that similar designs are good and is more likely to generate them in the future. Over many iterations, the controller learns which architectural choices lead to strong networks, without any human manually designing layers or connections.

5.3.3 Deepline

Deepline is an AutoML framework that uses deep reinforcement learning to build machine learning pipelines in a modular and scalable way. Pipelines are built by selecting components from six predefined primitive families:

1. Data pre-processing,
2. Feature pre-processing,
3. Feature selection,

4. Feature engineering,
5. Classification or regression,
6. Combiners.

The system employs a Dueling Deep Q-Network agent to learn a policy over pipeline construction steps, where each action corresponds to selecting a valid primitive at the current stage. A key innovation in Deepline is the hierarchical-step plugin. Instead of considering all possible primitives at once, the agent first selects a primitive family and then chooses a valid primitive within that family. This allows Deepline to handle a changing number of legal actions while keeping a fixed-size state representation, significantly accelerating training.

Example: Deepline works like assembling a recipe step by step. Instead of randomly choosing from thousands of tools, it first decides what kind of step is needed (for example, “data cleaning” or “classifier”). Only then it choose a specific tool that fits that step, such as a scaler or a random forest. By narrowing the choices at each step, Deepline learns much faster. This structured decision process makes the search efficient even when the number of possible pipeline components is very large.

6 Lecture 6: Meta-Learning and Transfer Learning

6.1 Introduction to Meta-Learning

While standard machine learning (such as classification or regression) focuses on generalizing across individual data points within a single dataset, **meta-learning** aims to generalize across different **datasets and tasks**.

6.2 Meta-Learning with Memory-Augmented Neural Networks (MANN)

In few-shot or zero-shot settings, where data is extremely rare, models must be capable of implementing a significant change in behavior from only a small number of examples. Memory-Augmented Neural Networks (MANNs) achieve this by combining two stages of learning:

1. **Slow Learning:** General knowledge is acquired gradually across many tasks using standard gradient descent to optimize parameters θ .
2. **Fast Adaptation:** Rapid, task-specific learning is enabled by an external memory module, which acts as a stable container of information that the model can query to adapt quickly to new situations.

To prevent the model from simply memorizing direct input–output mappings, MANNs employ a **label shuffling mechanism**. In this setup, an input sample x_t is presented at time t , but its corresponding label y_t is provided only at the next time step. This forces the network to store information about inputs in memory until the relevant label becomes available later in the sequence.

The system uses a controller network to interact with the **external memory** through **read and write operations**. To determine which memory locations to **read**, the controller generates a key vector k_t for input x_t . This key is compared to each memory row $M_t(i)$ using cosine similarity:

$$K(k_t, M_t(i)) = \frac{k_t \cdot M_t(i)}{\|k_t\| \|M_t(i)\|}. \quad (36)$$

The resulting similarity scores are normalized using a softmax function to produce a read-weight vector w_t^r .

For **writing** new information, MANNs use a Least Recently Used Access (LRUA) rule to ensure efficient memory utilization. The model writes either to the least-used memory location (to store new information) or to the most recently used location (to refine existing information).

MANNs have demonstrated strong performance on the Omniglot dataset, which contains 1,643 classes with only a few examples per class. Experiments show that MANNs achieve higher accuracy compared to standard LSTM and feedforward networks, highlighting their ability to learn effectively from limited data.

6.3 Simple Neural Attentive Meta-Learner (SNAIL)

The Simple Neural Attentive Meta-Learner (SNAIL) was designed to address a key limitation of earlier meta-learning systems: their hardcoded structure made it difficult to identify **which past experiences were most relevant** for the current decision. In simple terms, older models could remember the past, but struggled to decide what part from that past actually mattered. SNAIL overcomes this limitation by combining two mechanisms:

- **Temporal (Dilated) Convolutions:** These layers provide access to past information. Unlike standard convolutions, where accessing longer histories requires more parameters, dilated convolutions expand the temporal receptive field without increasing the number of parameters. Intuitively, this is similar to keeping short summaries of events from different points in time (recent, medium-term, and long ago), rather than remembering every single detail. This makes SNAIL scalable to long sequences.
- **Attention Models:** Attention mechanisms act as a focused searchlight over history, allowing the model to retrieve one specific, highly relevant past event regardless of how far back it occurred. In simple terms, attention lets the model “flip back” to the exact moment in the past that explains what is happening now, while ignoring irrelevant information.

6.3.1 Architectural Building Blocks

Attention Models: Attention models assign a weight to each past time step based on its relevance to the current decision. Instead of compressing all history into a single fixed representation, attention computes a weighted combination of past states, allowing the model to focus on the most informative events. In simple terms, attention answers the question: which moment in the past matters most right now? This mechanism enables context-dependent memory access and is especially effective when only a small subset of past experiences is critical for solving the current task.

SNAIL is composed of three main architectural components:

1. **Dense Blocks:** These blocks rollback the current input with a summary representation of all previous states. This allows information from the past to directly influence current decisions, similar to taking notes from previous steps and keeping them visible while solving the current problem.
2. **Temporal Convolution Layers:** Multiple dense blocks are stacked together to form a single temporal stream that aggregates historical information at different time scales, creating an organized timeline of past experience.
3. **Attention Blocks:** The final stage applies attention over this entire temporal stream to select the most relevant pieces of history and use them to determine the action or prediction. This is analogous to scanning through all notes and highlighting only the few lines that directly answer the current question.

SNAIL exhibits two key strengths. First, it is **consistent**, meaning its performance improves reliably as more data becomes available. Second, it is **expressive**, as it can represent a wide range of learning and decision-making strategies. However, SNAIL lacks explicit mechanisms for structured exploration and is generally less efficient than some specialized off-policy meta-RL algorithms.

6.4 Model-Agnostic Meta-Learning (MAML)

The primary challenge in few-shot learning is that models are highly vulnerable to overfitting when only a very small amount of data is available. MAML addresses this challenge by designing a model for **fast adaptation**. Instead of learning task-specific solutions, MAML learns parameters θ that serve as an initialization, which can be quickly specialized to a new task using only a small number of gradient updates.

MAML can be understood as learning a good starting point rather than a complete solution. The goal is not to solve every task optimally in advance, but to find parameters from which learning a new task becomes fast.

Unlike standard supervised learning, which optimizes parameters for immediate performance on a single dataset, MAML optimizes for performance after adaptation. The objective is to find parameters θ such that small gradient updates lead to large improvements on a new task.

Example: Consider a robot that must operate under varying conditions such as different terrains or payloads. Training a separate controller from scratch for each condition would be inefficient. Instead, MAML learns an initialization that captures the shared structure of these tasks. When a new condition is encountered, the controller adapts using only a few gradient updates, allowing the robot to regain good performance almost immediately.

6.4.1 Architectural Building Blocks

In reinforcement learning, MAML enables policies to adapt rapidly to changing dynamics through an iterative procedure:

1. **Sampling:** A batch of tasks T_i is sampled from a task distribution $p(T)$.
2. **Inner Loop (Adaptation):** For each task, K trajectories are sampled using the current policy f_θ . Task-specific parameters are computed via gradient descent:

$$\theta'_i = \theta - \alpha \nabla_\theta \mathcal{L}_{T_i}(f_\theta). \quad (37)$$

3. **Outer Loop (Meta-Update):** New trajectories are collected using the adapted policy $f_{\theta'_i}$. The meta-parameters are then updated as:

$$\theta \leftarrow \theta - \beta \nabla_\theta \sum_{T_i \sim p(T)} \mathcal{L}_{T_i}(f_{\theta'_i}), \quad (38)$$

where β is the meta step size.

MAML offers several advantages over recurrent meta-learning approaches:

- **Consistency:** Since MAML relies on gradient descent, it is guaranteed to converge to at least a local optimum, unlike recurrent models whose convergence behavior is less predictable.
- **Versatility:** MAML is model-agnostic and can be applied to any architecture trained via gradient-based optimization.
- **Real-Time Adaptation:** When applied to dynamics models (e.g., GrBAL), MAML enables robots to adapt effectively to unexpected changes such as terrain variation or component failure.
- **Trade-offs:** Although MAML is expressive and consistent, it lacks structured exploration and is generally not an off-policy algorithm.

6.5 Real-Time Adaptation

Applying Deep Reinforcement Learning (DRL) to physical systems is challenging because data collection is expensive and policies trained for a specific environment often fail under unexpected environmental changes, such as wind, changes in terrain, or hardware failures.

Meta-RL addresses these challenges by learning a **dynamics model prior** that can be rapidly adapted to the local environment using recent experience. This approach relies on three key assumptions:

1. **Non-Stationarity:** Each time step may correspond to a slightly different environment.
2. **Common Structure:** Although environments differ, they share an underlying structure that can be captured by a meta-learned prior.
3. **Local Consistency:** Over short time horizons, the dynamics are assumed to be approximately consistent, enabling effective adaptation.

6.5.1 Adaptation Algorithms

The adaptation framework learns a general **dynamics prior** in advance and then adjusts it online using recent experience. Intuitively, the robot first learns a model of how its actions affect the world. During deployment, it observes a short window of recent states and actions $\{s_{t-M:t}, a_{t-M:t}\}$ and uses this information to quickly update its model so it matches the current environment. This allows the system to adapt in real time to changes. Two adaptation algorithms are:

- **ReBAL (Recurrent-Based Adaptation Learner)**

ReBAL performs adaptation using a recurrent neural network (RNN) that directly processes recent state-action histories. The RNN implicitly infers the current environment dynamics and outputs adapted model parameters without performing explicit gradient updates. The Dynamics RNN receives a rolling window of recent observations and actions and encodes them into a hidden state, which represents the current local dynamics. This hidden state conditions the dynamics model used for planning and control.

Example: Imagine a robot that suddenly starts walking on sand instead of solid ground. ReBAL detects the change by observing recent slippage in its motion and implicitly adjusts its internal dynamics model without explicitly computing gradients to compensate for the softer terrain.

ReBAL adapts very quickly and is computationally efficient at runtime. However, because adaptation is implicit and learned through recurrence, it may struggle to generalize to dynamics changes that differ significantly from those seen during training.

- **GrBAL (Gradient-Based Adaptation Learner)**

GrBAL performs adaptation using explicit gradient-based updates via the MAML framework. Instead of relying on recurrence, it treats the dynamics model as a learnable function that can be rapidly fine-tuned using a small amount of recent data. A meta-trained prior θ^* serves as a good initialization. When new data arrives, the model performs one or a few gradient descent steps to produce adapted parameters θ' , explicitly optimizing the prediction of the recent transitions.

Example: Consider a quadruped robot that loses one of its legs. GrBAL uses a few recent steps to compute gradients that adjust the dynamics model, allowing the robot to quickly relearn how its body moves and continue walking effectively.

Experimental results show that GrBAL significantly outperforms non-adaptive baselines in challenging scenarios. In the Missing Leg experiment, the GrBAL-adapted model maintains a high return, while the performance of a non-adaptive model drops nearly to zero. This demonstrates that meta-learning a dynamics prior enables robots to rapidly re-learn their own dynamics when physical conditions change.

Comparison of Architectures: The following table summarizes how different meta-learning architectures satisfy these desired properties:

Property	SNAIL	MAML-PG	ReBAL
Consistent	Yes	Yes	Yes
Expressive	Yes	Yes	Yes
Structured Exploration	No	No	No
Efficient & Off-policy	No	No	Yes

6.6 Forward Transfer Learning

Transfer learning refers to reusing knowledge acquired from a source task or dataset to accelerate learning and improve performance on a target task. This approach is especially valuable when training from scratch is computationally expensive, unstable, or when only limited data is available. Unlike **meta-learning**, which explicitly learns how to adapt quickly to entirely new tasks, transfer learning typically adapts a model that was trained for a specific source task by fine-tuning or reusing its representations for a related target task.

Forward transfer learning refers to learning on a source task and transferring the acquired knowledge to a target task. There are two primary approaches:

6.6.1 Finetuning and Freezing

A model pre-trained on a source task is further trained on a target task. In deep learning, a common strategy is to **freeze** the lower layers of the network, which typically capture general features, and retrain only the final layers. This approach is particularly effective when the target dataset is small and tend to overfitting.

6.6.2 Training for Diversity (Maximum Entropy RL)

To improve robustness and transferability, models can be trained to learn **diverse behaviors** rather than a single optimal solution. This is achieved using **Maximum Entropy Reinforcement Learning**, which encourages policies to maximize both reward and entropy. By learning multiple valid ways to solve a task, the resulting policy provides a stronger and more flexible starting point for adaptation.

The Maximum Entropy RL objective is defined as:

$$\pi_{\text{MaxEnt}}^* = \arg \max_{\pi} \sum_t \mathbb{E}_{(s_t, a_t) \sim p^{\pi}} [r(s_t, a_t) + \alpha \mathcal{H}(\pi(\cdot | s_t))], \quad (39)$$

where α controls the trade-off between reward maximization and exploration. In practice, this objective is often implemented using energy-based models and soft Q -functions.

6.6.3 Progressive Networks

To prevent **catastrophic forgetting** (the loss of previously learned tasks when learning new ones), **Progressive Neural Networks** assign each task its own network column. When a new task is introduced, all previous columns are frozen, and a new column is added with **lateral connections** to earlier columns.

This architecture allows the model to:

- Retain performance on all previously learned tasks.
- Accelerate learning on new tasks using transferred knowledge.
- Solve multiple independent tasks within a single system.

The hidden representation for task k at layer i is given by:

$$h_i^{(k)} = f \left(W_i^{(k)} h_{i-1}^{(k)} + \sum_{j < k} U_i^{(k:j)} h_{i-1}^{(j)} \right), \quad (40)$$

where lateral connections allow information flow from earlier task columns.

6.6.4 Self-Supervision in Reinforcement Learning

Transfer learning can also be improved through **self-supervised learning**, where agents learn useful representations without explicit labels. This is often done by training an encoder using supporting learning tasks, discarding the decoder, and reusing the encoder for downstream tasks.

Common supporting learning tasks include:

- **Reward Binning:** Predicting whether the next reward will be positive or negative.

- **Dynamics Prediction:** Predicting the next state s_{t+1} from (s_t, a_t) .
- **Inverse Dynamics:** Predicting the action that caused a state transition.
- **Reconstruction:** Reconstructing inputs using models such as variational autoencoders.

Example: A robot explores an environment without knowing the task yet. By learning to predict how its actions move it (dynamics) or which action caused a movement (inverse dynamics), it builds a useful internal representation that can later be reused for navigation, manipulation, or control tasks.

6.7 Multi-Task Transfer Learning

Multi-task transfer learning trains a single model on multiple related tasks simultaneously, encouraging the learning of shared representations across tasks. By solving several tasks together rather than independently, the model can reuse common structure, leading to improved generalization, faster learning, and better data efficiency.

When multiple source tasks are available, transfer performance can be further improved by **combining knowledge from diverse domains**. Instead of learning a separate model for each task, multi-task approaches exploit similarities between tasks to build a unified and more robust representation.

6.7.1 Model-Based Reinforcement Learning

In model-based reinforcement learning, transfer occurs at the level of the **dynamics model** rather than the policy. The key assumption is that while tasks may differ in objectives or rewards, the underlying physical dynamics of the environment remain consistent. By learning a shared dynamics model, the agent can quickly adapt to new tasks through planning or re-optimization.

Example: A robot first learns how its joints, motors, and mass behave. This learned dynamics model can then be reused for walking, turning, or carrying objects, with only the planner or task objective changing, rather than retraining the entire system from scratch.

6.7.2 Actor-Mimic Approach

The Actor-Mimic approach transfers knowledge by training a single **multi-task network** to imitate multiple expert policies, each trained on a different task. Instead of learning directly from environment rewards, the network learns by matching the experts' Q -values using policy regression and, optionally, feature regression to align internal representations. This promotes a shared feature space across tasks.

Example: Several expert agents are trained to play different Atari games. The Actor-Mimic Network learns from all experts at once, producing a shared internal representation that enables faster learning when adapting to a new game.

6.7.3 Model Distillation

Model distillation compresses knowledge from a large or ensemble model into a smaller student network using soft targets. Unlike hard labels, soft targets encode similarity information between outputs, allowing the student to capture richer structure learned by the teacher.

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}, \quad (41)$$

where $T > 1$ controls the softness of the output distribution.

Example: An ensemble of accurate but computationally expensive classifiers is trained offline. Their predictions are distilled into a compact network that runs much faster while retaining most of the original performance.

6.7.4 Modular Neural Networks

Modular neural networks decompose policies into **task-specific modules** and **robot-specific modules**. Task modules encode *what* needs to be done, while robot modules encode *how* actions are executed. This modular structure enables systematic reuse and recombination of skills across tasks and agents.

Example: A grasping module learned on one robotic arm can be combined with a locomotion module from another robot, enabling a new robot–task combination to succeed without retraining the entire policy.

7 Lecture 7: Dealing with Large State and Action Spaces

7.1 Large Discrete Action Spaces

Applying Deep Reinforcement Learning (DRL) to real-world domains often requires operating in environments with a massive number of discrete actions. Standard DRL methods struggle in these scenarios because they cannot effectively generalize across similar actions and often suffer from poor computational scaling.

7.2 The Wolpertinger Policy

To improve performance, a key solution is to **embed discrete actions into a continuous space**, where their relationships can be modeled and generalized. The Wolpertinger architecture is a specialized actor–critic framework designed to handle extremely large discrete action spaces by decoupling the policy network from the Q -function. Action selection proceeds in three stages:

1. **Proto-action Generation:** The actor network f_{θ^π} receives the current state s and produces a proto-action $\hat{a}$ in a continuous embedding space. This proto-action represents a latent preference for a desired behavior, but does not necessarily correspond to a valid discrete action.
2. **K-Nearest Neighbors (KNN):** Since $\hat{a}$ may not be a legal discrete action, the system retrieves a subset $\mathcal{A}_k$ of k valid discrete actions that are closest to $\hat{a}$ according to Euclidean distance.
3. **Critic Refinement:** The critic Q_{θ^Q} evaluates each action in $\mathcal{A}_k$ and selects the one with the highest expected return:

$$a = \arg \max_{a_j \in \mathcal{A}_k} Q_{\theta^Q}(s, a_j).$$

This step is critical for success, as an **action that is geometrically close in the embedding space may still have a low value** under the true Q -function.

The main disadvantages of the Wolpertinger policy arise from the use of a **K-Nearest Neighbors (KNN)** step between the actor and the critic. Since KNN is not differentiable, gradients cannot flow directly from the critic back to the actor. As a result, the model cannot be trained in a fully end-to-end manner, which reduces training efficiency. Another drawback concerns the **action embeddings**. If the action features or their representation change, all action embeddings must be recomputed. This process can be computationally expensive and limits the scalability of the method in large or dynamically changing action spaces.

7.3 Generative Proto-Sequence (GPS)

Traditional Deep Reinforcement Learning (DRL) often struggles in complex environments characterized by large state spaces, long action horizons, and sparse rewards. A key reason is the decision bottleneck, where selecting actions one step at a time leads to inefficient exploration and **poor credit assignment** (difficulty in linking final outcomes to earlier decisions). To address these limitations, the **Generative Proto-Sequence (GPS)** architecture was developed to generate and evaluate entire action sequences in a single step.

7.3.1 Architectural Building Blocks

GPS is an actor-critic architecture composed of three main components:

- **The Actor (Policy):** The actor receives the current state observation s_t and outputs a **proto-sequence embedding** k . This latent vector represents a complete multi-step behavioral strategy rather than a single action.
- **Proto-Sequence Decoder (PSD):** The decoder is implemented as a **Variational Autoencoder (VAE)**, pre-trained on diverse synthetic action sequences. A **VAE** is a generative model that learns a continuous latent space from which new samples can be drawn, allowing it to generate coherent and diverse outputs. Given the embedding k , the decoder samples from this latent space to produce a variable-length sequence of executable actions $(a_t, a_{t+1}, \dots, a_{t+n})$ in a single forward pass. This is significantly more efficient than other approaches that generate sequences step by step.
- **The Critic (Evaluation):** Unlike standard critics that evaluate individual state-action pairs, the GPS critic evaluates the current state s_t together with the entire decoded action sequence A . It outputs a single **Q-value** representing the expected cumulative reward of executing the full plan.

GPS uses a **fully differentiable, end-to-end pipeline** in which gradients propagate from the critic, through the Proto-Sequence Decoder, and back to the actor. Unlike standard actor-critic methods that evaluate and optimize single-step actions, the GPS critic evaluates the entire decoded action sequence and assigns a single Q -value to the full plan. As a result, the actor is trained indirectly: instead of maximizing a policy gradient objective, it **learns to produce a latent embedding k that decodes into a high-value sequence predicted by the critic**.

This mechanism differs from a standard actor-critic framework, where the critic provides a scalar value estimate for a chosen action but does not influence how future action sequences are generated.

To further improve data efficiency, GPS **learns not only from the full executed trajectory, but also from shorter parts of it**. By using both the beginning (prefixes) and the ending (suffixes) of a trajectory, the agent can obtain multiple learning signals from a single rollout. This helps the model better understand which decisions contributed to success or failure, leading to improved credit assignment.

Evaluations in maze environments show that GPS offers several advantages over traditional methods such as DQN:

- **Faster Convergence:** GPS converges faster than baseline methods. For example, in a 16×16 maze, GPS achieved a success rate of 0.74 after 100,000 steps, while DQN reached only 0.192.
- **Enhanced Exploration:** By committing to coherent multi-step strategies instead of selecting actions greedily at each step, GPS explores the state space more effectively.
- **Generalization and Robustness:** GPS learns transferable macro-behaviors and can generate novel action sequences that were not explicitly seen during decoder training. Sequence-level planning also improves robustness to stochastic dynamics, such as random or “sticky” actions, by buffering execution noise.

7.4 Action Elimination (AE-DQN)

Large action spaces introduce a major challenge for Q-learning, often causing the algorithm to converge to sub-optimal policies because the agent wastes significant effort exploring irrelevant or consistently poor actions. While techniques such as reward shaping attempt to discourage such actions, they are difficult to tune and sample-inefficient, since the agent must still explore an action before learning to avoid it. **Action Elimination (AE-DQN)** addresses this issue by explicitly pruning the action space, allowing the agent to focus its learning on high-value action candidates.

7.4.1 Architectural Building Blocks

The key idea behind AE-DQN is to decouple action elimination from the main Markov Decision Process (MDP). This is achieved by training two separate but interacting components:

- **Action Elimination Network (AEN):** A neural network whose purpose is to identify actions that are likely to be sub-optimal. The final hidden layer of the AEN is used as input to a **contextual multi-armed bandit**, which decides which actions should be eliminated.
- **Standard DQN:** A conventional Deep Q-Network that performs reinforcement learning, but selects actions only from a reduced action set A' , as recommended by the AEN.

7.4.2 Elimination Mechanism and Exploration

Action elimination proceeds through an iterative process:

1. **Bandit Filtering:** Every L iterations, the contextual bandit uses the AEN representations to identify and remove actions that are predicted to be sub-optimal. The resulting filtered action set is passed to the DQN.
2. **Loss Propagation:** The elimination decision produces an elimination signal e_t , which is fed back into the AEN to compute a loss and update its parameters. This allows the AEN to improve its action filtering over time.
3. **Robust Exploration:** Action selection follows an ϵ -greedy strategy. Even actions that have been eliminated retain a small probability of being selected, ensuring that mistakenly eliminated actions can be re-evaluated and recovered.

The effectiveness of AE-DQN is demonstrated on the Troll Quest benchmark. In this environment, an AE-DQN operating over 215 actions converges significantly faster and achieves much higher cumulative reward than a standard DQN with the same action space. Its performance nearly matches that of a DQN trained in a much smaller 35-action environment, highlighting the benefit of explicit action pruning.

7.5 Hierarchical Reinforcement Learning (HRL)

Hierarchical Reinforcement Learning (HRL) is **suited for environments with sparse rewards** and very large state-action spaces, where simple exploration strategies such as ϵ -greedy are often insufficient. HRL guides exploration by introducing **intrinsic goals**, allowing the agent to decompose a complex task into a sequence of manageable sub-tasks.

7.5.1 Hierarchical DQN (h-DQN)

The h-DQN framework structures learning into two distinct levels of control that operate at different temporal scales:

- **Meta-Controller:** Receives the environment state s_t and selects a high-level goal $g_t \in \mathcal{G}$ from a predefined goal space. It is trained to maximize **extrinsic rewards** F_t provided directly by the environment.
- **Controller:** Receives both the state s_t and the goal g_t , and selects primitive actions a_t . The controller attempts to achieve the current goal for a fixed number of steps or until the goal is reached. It is trained using **intrinsic rewards** $R_t(g_t)$ generated by an internal critic that evaluates whether the selected goal has been successfully accomplished.

This hierarchical structure is highly effective in environments with extremely delayed rewards. For example, in the Atari game Montezuma’s Revenge, where standard DQN agents typically score zero, h-DQN achieves significantly higher performance by learning to reach meaningful sub-goals such as “reaching the key” before attempting to complete the full level.

7.5.2 Feudal Networks (FuN)

Feudal Networks (FuN) extend the hierarchical idea by introducing a **Manager–Worker** architecture with strictly top-down communication and different resolutions:

- **Manager:** Operates at a high-level resolution (e.g., every c steps) and does not act on the environment directly. Instead, it produces a goal vector g_t in a learned latent state space, representing a desired direction of change in the environment.
- **Worker:** Operates at every time step and executes primitive actions a_t . The worker is not trained on external rewards. Instead, it receives an **intrinsic reward** based on the cosine similarity between the actual state change and the manager’s goal vector:

$$r_{\text{in}} = \frac{1}{c} \sum_{i=1}^c d_{\cos}(S_{t+i} - S_t, g_t). \quad (42)$$

Feudal Networks provide an effective solution to long-term credit assignment. In domains such as banking, if a customer leaves months after an aggressive marketing campaign, a flat agent struggles to connect this failure to the earlier decisions that caused it. In Feudal Networks, the **Manager** receives the negative reward for choosing an overly aggressive long-term goal, ensuring that the mistake is attributed to the strategic decision rather than to low-level actions.

Moreover, the hierarchical structure allows the agent to reason about hidden factors, such as a customer’s unobserved “patience.” A greedy **Worker** may focus on obtaining immediate clicks and quickly exhaust this resource, while the **Manager** can learn to issue more conservative or “rest” goals. This enables the system to balance short-term gains with long-term customer retention.

h-DQN vs. Feudal Networks Hierarchical DQN (h-DQN) and Feudal Networks (FuN) both introduce hierarchical control to handle sparse rewards and long-term dependencies, but they differ in how goals are represented and learned. h-DQN uses **discrete, predefined goals** and separates learning into a meta-controller trained on extrinsic rewards and a controller trained using manually designed intrinsic rewards. In contrast, Feudal Networks employ **continuous, learned goal vectors** in a latent state space and define intrinsic rewards using cosine similarity between desired and actual state changes.

7.6 Advanced Scaling Architectures

7.6.1 Branching Dueling Q-Networks (BDQ)

Branching Dueling Q-Networks (BDQ) extend the Dueling DQN architecture to handle **high-dimensional action spaces**, such as those used to control robotic joints. The architecture employs a shared **state-value estimator** that branches into N independent **advantage estimators**, one for each action dimension. This design allows the model to decompose a complex action into largely independent sub-decisions while efficiently identifying redundancies across dimensions.

The state-value and advantage components are combined through an aggregation layer. Learning is stabilized by averaging the temporal-difference (TD) error across all branches. The loss function is defined as:

$$L = \mathbb{E}_{(s,a,r,s') \sim \mathcal{D}} \left[\frac{1}{N} \sum_{d=1}^N (y_d - Q_d(s, a_d))^2 \right], \quad (43)$$

where y_d is the target value for branch d , Q_d is the branch-specific Q-value, and $\mathcal{D}$ denotes the prioritized experience replay buffer.

Example: Consider a robotic arm with multiple joints, where each joint can move independently. Instead of treating every combination of joint movements as a separate action, BDQ assigns one branch per joint. Each branch selects the best movement for its joint, allowing the robot to construct a full action efficiently without enumerating all joint combinations. The general idea is **one state-value function for the complete robotic arm, but different advantage functions for each joint**.

7.6.2 Jointly-Learned State–Action Embedding

While many reinforcement learning approaches learn state and action embeddings independently, this architecture proposes **jointly learning** both to better capture the interaction between states and actions. Joint learning improves sampling efficiency and generalization in very large state–action spaces.

The model learns a continuous state embedding space $\mathcal{X}$ and a continuous action embedding space $\mathcal{E}$. A transition model T predicts the next state s_{t+1} from the concatenation of the embedded state and action. The projection functions ϕ (for states) and g (for actions) are optimized by minimizing the KL-divergence between predicted and true environment transitions. In addition, a mapping function f reconstructs actions from the latent action embedding by minimizing reconstruction error.

Example: In a navigation task, similar states (e.g., nearby locations) and similar actions (e.g., turning slightly left or right) are embedded close together. By learning these embeddings jointly, the agent can generalize its experience: an action that worked in one state can be adapted to similar states without explicit retraining.

7.6.3 Value Iteration Networks (VIN)

Value Iteration Networks (VIN) integrate a differentiable **planning module** into an actor–critic framework, enabling agents to reason over long horizons in large environments such as complex mazes. The core component is a Value Iteration (VI) module that predicts action values multiple steps into the future.

The VI module performs approximate value iteration using convolutional layers that represent learned reward maps $\bar{R}$ and transition probabilities $\bar{P}$, followed by channel-wise max-pooling to select the most optimistic outcomes. Both the reward function f_R and the transition function f_P are learned end-to-end through backpropagation. This built-in planning capability allows VINs to achieve substantially higher success rates in large-scale maze navigation compared to standard CNNs or fully connected networks. It can be seen as an Actor-critic architecture but in this case the critic provides his forecast for a few steps ahead (planning) inside of just one.

Example: In a maze navigation task, a VIN learns an internal map of the maze and repeatedly plans several steps ahead before acting. Instead of reacting only to the current observation, the agent can anticipate dead ends and choose actions that lead toward the goal more reliably.

8 Lecture 8: Advanced Model Learning and Exploration

8.1 Learning in Latent Space

In most real-world Reinforcement Learning scenarios, the true states of the environment are unknown, and the agent only observes **observations** (e.g., camera images or sensor readings), which provide only incomplete information about the true state. As a result, the agent must learn to infer useful state representations from observations rather than relying on direct state access. To handle this, agents can either apply model-free methods directly on observations or learn a compact **latent-space representation** $g(O)$ that captures the essential structure of the environment.

8.1.1 Model-Free Latent Learning

In a model-free latent learning approach, the agent learns a control policy directly from observations, without explicitly modeling the environment dynamics or underlying state. The key idea is to first compress inputs into a compact representation and then apply standard reinforcement learning on this reduced space.

1. **Data Collection:** An exploratory policy is used to collect a dataset of observations from interaction with the environment.
2. **Dimensionality Reduction:** An autoencoder is trained in an unsupervised manner to compress high-dimensional observations into a low-dimensional latent feature space. This latent space serves as a compact representation of the input.

3. **Policy Training:** A model-free reinforcement learning algorithm (e.g., Q-learning or DQN) is trained on the latent features to map compressed observations directly to actions.

This approach significantly reduces the dimensionality of the learning problem and enables efficient processing of inputs. However, because the autoencoder is trained independently of the control objective, there is no guarantee that the learned latent representation captures task-relevant or control-critical features. As a result, learning can be sample-inefficient and convergence may be slow, especially in complex or high-dimensional environments.

8.1.2 Model-Based Latent Learning

For complex robotic tasks such as object manipulation, the agent does not have access to the true state of the environment. Learning both perception and control simultaneously from scratch is slow and unstable. To reduce training time, model-based approaches often **train vision and control separately** until each reaches a reasonable baseline performance. A well-known example is the **deep visuomotor policy** architecture used for robot manipulation tasks such as grasping tools or pushing objects.

Visuomotor Policy Architecture Unlike standard convolutional neural networks used for image classification, this architecture is explicitly designed for **control from vision**. The key modifications are:

- **No Pooling:** Pooling layers are intentionally removed because they destroy exact spatial information. For manipulation tasks, the agent must know where an object is located in the image, not just whether it exists.
- **Spatial Softmax:** Instead of pooling, a spatial softmax is applied independently to each convolutional feature map. This transforms each feature map into a probability distribution over pixel locations and outputs the expected 2D coordinate of the feature. As a result, the network extracts a small set of precise, task-relevant keypoints that summarize the visual scene in a compact and control-friendly form.
- **Fully Connected Control Layers:** The 2D visual keypoints are concatenated with low-dimensional robot state information (e.g., joint angles or end-effector pose). This combined representation is passed through fully connected layers that map perception directly to motor commands.

Training alternates between two objectives:

- Optimizing the policy parameters θ to improve task performance.
- Optimizing the learned dynamics parameters τ that model how actions affect observations.

Because true environment states are unavailable, rewards are computed by comparing the current camera image to a **goal image**. The closer the current observation is to the goal image, the higher the reward. This structured latent representation enables efficient learning of complex visuomotor skills while preserving critical spatial information. However, as in other latent-space approaches, there is no formal guarantee that the learned visual features fully capture the true task-relevant aspects of the environment.

8.2 Generative Models and Variational Learning

Generative models are used to **learn how data looks**, rather than only how to act. In reinforcement learning, this means learning the distribution of the agent’s observations, e.g., images from a camera, so the agent can generate or predict what it might see next. Many generative models assume that each observation is produced from a smaller, hidden representation called a **latent variable**. In practice, however, computing the exact probability of an observation by considering all possible latent variables is extremely difficult and computationally expensive. **Variational learning** provides a practical solution to this problem. Instead of trying to compute the true probability exactly, the model learns a good approximation that is easier to optimize. This approximation is trained using neural networks and allows the model to efficiently learn useful latent representations from data.

A common example of this idea is the **Variational Autoencoder (VAE)**. A VAE learns to compress high-dimensional inputs, such as images, into a compact latent space and then reconstruct them back. At the same time, it enforces a structured and smooth latent space, which makes it suitable for prediction, planning, and model-based reinforcement learning.

Variational Autoencoder (VAE) Architecture implements this approach using a specific neural network architecture:

- **Encoder** $q_\theta(z | x)$: Unlike a standard autoencoder that maps an input to a single deterministic point, the VAE encoder outputs the parameters of a **stochastic Gaussian probability distribution** in a lower-dimensional latent space.
- **Sampling**: A latent variable z is sampled from the predicted distribution.
- **Decoder** $p_\phi(x | z)$: The decoder takes the sampled latent variable z and outputs the parameters required to reconstruct the original input x .

The objective of the network is to minimize the following loss function:

$$\ell_i = -\mathbb{E}_{z \sim q_\theta(z|x_i)} [\log p_\phi(x_i | z)] + D_{KL}(q_\theta(z | x_i) \parallel p(z)). \quad (44)$$

This loss function consists of two critical components:

1. **Reconstruction Loss**: The first term uses the negative log-likelihood of the input features (e.g., pixels) to ensure that the reconstructed output matches the original observation.
2. **KL-Divergence Regularizer**: The second term measures the difference between the encoder’s predicted distribution and a prior distribution, typically $p(z) = \mathcal{N}(0, 1)$. This term prevents it from simply memorizing the training data.

The goal is to find a bound that closely approximates the true likelihood of x while remaining computationally tractable. The core insight of the VAE framework is that a neural network can be trained to predict the parameters of this approximating distribution.

8.3 Embed to Control (E2C)

Embed to Control (E2C) is a model-based reinforcement learning approach designed to address the challenge of learning control policies directly from high-dimensional pixel observations. It does so by **jointly learning** a low-dimensional latent embedding and a transition model defined in that latent space. Unlike earlier approaches that learn representations and dynamics separately, E2C optimizes the entire system end-to-end using transition triplets $\mathcal{D} = \{o_t, a_t, o_{t+1}\}$ collected from interaction with the environment.

8.3.1 Architectural Building Blocks

The E2C framework uses variational autoencoders (VAEs) to map observations into a latent space in which the dynamics are easier to model. The architecture consists of three main components:

- **Encoder** (g): Represented by the distribution $q_\phi(z | o_t)$, the encoder maps the high-dimensional observation o_t to a stochastic latent embedding z_t .
- **Latent Transition Model**: Represented by $\hat{q}_\psi(\hat{z}_{t+1} | z_t, a_t)$, this model predicts the next latent state $\hat{z}_{t+1}$ given the current latent state z_t and the applied action a_t .
- **Decoder**: Represented by $p(o | z)$, the decoder maps latent states back into the observation space to reconstruct the predicted next observation $\hat{o}_{t+1}$.

The system is trained by minimizing a joint loss function $\mathcal{L}(\mathcal{D})$ that balances accurate image reconstruction with consistent latent dynamics:

$$\mathcal{L}(\mathcal{D}) = \sum_{o_t, a_t, o_{t+1}} [L_{\text{bound}}(o_t, a_t, o_{t+1}) + \lambda D_{KL}(\hat{q}_{\psi}(\hat{z}_{t+1} \mid z_t, a_t) \parallel q_{\phi}(z_{t+1} \mid o_{t+1}))]. \quad (45)$$

The first term, L_{bound} , corresponds to the **variational lower bound** and ensures that the latent representation can faithfully reconstruct observations. The second term is a **KL-divergence penalty** that enforces consistency between the predicted next latent state $\hat{z}_{t+1}$ and the encoded latent state z_{t+1} of the actual next observation. By learning a coherent latent dynamics model, E2C enables the agent to simulate future trajectories entirely in latent space without additional interaction with the real environment. This capability has been demonstrated in domains such as visual cart-pole control and robotic arm manipulation, where the model can predict long sequences of future frames from a single initial image and a sequence of proposed actions.

8.4 Action-Conditional Video Prediction

Unlike methods that rely on abstract latent embeddings, action-conditional video prediction models operate directly in **pixel space**. By learning to predict how the environment’s visual appearance changes in response to specific actions, these models allow agents to perform planning and informed exploration without requiring an explicit state representation.

A major challenge in video prediction is the **compounding error problem**. When a model is trained only on one-step predictions, small errors in the first predicted frame are fed back as input for subsequent predictions. As a result, errors accumulate rapidly, leading to unrealistic and diverging trajectories over time. To address this issue, the model is trained using a **multi-step prediction loss**, which penalizes prediction errors over an entire horizon of K steps:

$$\mathcal{L}_K(\theta) = \frac{1}{2K} \sum_i \sum_{k=1}^K \|\hat{s}_i^k - s_i^k\|^2. \quad (46)$$

By evaluating performance over a full sequence rather than a single frame, this objective encourages the network to learn more stable and globally consistent dynamics.

8.4.1 Curriculum Learning Strategy

Directly training a model to predict K steps into the future is computationally difficult and often results in unstable gradients. To overcome this, a **curriculum learning** strategy is employed:

- **Incremental Horizons:** Training begins with very short prediction horizons, where the task is easier and more stable.
- **Hybrid Trajectories:** As training progresses, the model gradually increases the prediction horizon by combining its own predicted frames with segments from the ground-truth trajectory.

8.4.2 Architectural Building Blocks

Two main architectural designs are commonly used for action-conditional video prediction:

- **Feedforward Encoding:** A direct mapping from the current frame and action to the next predicted frame. This approach is simple but limited in capturing long-term temporal dependencies.
- **Recurrent Encoding:** Uses recurrent structures such as LSTMs or GRUs to maintain a temporal history, making it more effective for modeling moving objects and partially observable environments.

8.5 Informed Exploration

While standard ϵ -**greedy exploration** provides a simple and effective baseline, it is often slow and inefficient in high-dimensional environments. **Informed exploration** improves efficiency by using video-prediction models to guide exploration toward states that are visually unfamiliar, rather than selecting actions uniformly at random.

The agent maintains a **trajectory memory**

$$\mathcal{D} = \{s_i\}_{i=1}^d,$$

which stores the d most recently observed frames in a sliding window. When the agent decides to explore (instead of exploit), it uses its predictive model to generate a predicted future frame $s^{(a)}$ for each candidate action a . The unfamiliarity of a predicted state is measured using an estimated **visit frequency** $n_{\mathcal{D}}(s^{(a)})$, computed by comparing the predicted frame to frames stored in the trajectory memory:

$$n_{\mathcal{D}}(s^{(a)}) = \sum_{i=1}^d k(s^{(a)}, s_i). \quad (47)$$

The similarity kernel $k(x, y)$ is defined as:

$$k(x, y) = \exp\left(-\sum_j \frac{\max((x_j - y_j)^2 - \delta, 0)}{\sigma}\right), \quad (48)$$

where δ acts as a threshold to ignore minor pixel-level noise and σ controls the sensitivity of the similarity measure. During exploration, instead of choosing an action at random, the agent selects the action that minimizes the predicted visit frequency:

$$a_t = \arg \min_a n_{\mathcal{D}}(s^{(a)}). \quad (49)$$

This strategy encourages the agent to explore states that are most different from its recent experience.

The main idea of **informed exploration** is to select actions that lead to states which are *similar to the current observation*, but have been *rarely visited before*. This encourages exploration that is gradual and meaningful, avoiding both random jumps to unrelated states and repeated visits to already well-explored regions of the state space.

- **Advantages:** Informed exploration leads to significantly better coverage of the state space. For example, in games such as *Ms. Pacman*, visitation heat maps show that the agent explores a much wider range of locations compared to standard random exploration.
- **Limitations:** While predicting future frames is feasible in controlled environments such as Atari, generating accurate predictions in complex, stochastic real-world settings is considerably more difficult. In addition, there is no universal notion of visual similarity, which limits the applicability of this approach for long-horizon planning.

Example: The main idea of **informed exploration** is to select actions that lead to states which are **similar to the current observation**, but have been **rarely visited before**. This encourages exploration that is gradual and meaningful, avoiding both random jumps to unrelated states and repeated visits to already well-explored regions of the state space.

8.6 Inverse Reinforcement Learning on Images

Most approaches discussed previously rely on reconstructing full images to make predictions or decisions. However, reconstructing high-dimensional real-world images is computationally expensive and often unnecessary, since large portions of an image (e.g., background regions) are usually irrelevant to the task. Instead, an **inverse model** focuses directly on the relationship between observations and actions, without explicitly modeling the full image dynamics.

8.6.1 Inverse and Forward Models

The system uses two complementary models to capture environment dynamics without reconstructing full images:

1. **Inverse Model (G):** Given a starting state s_t and a resulting state s_{t+1} , the inverse model predicts the action $\hat{a}$ that most likely caused the transition:

$$\hat{a} = G(s_t, s_{t+1}; W_{\text{inv}}). \quad (50)$$

The inverse model provides the actions required to move from a current state toward a desired goal state.

2. **Forward Model (F):** Given the current state and an action, the forward model predicts the next state representation:

$$\hat{s}_{t+1} = F(s_t, a_t; W_{\text{fwd}}). \quad (51)$$

The forward model operates in the learned representation space and is used to rank or prioritize actions when multiple actions could lead toward the same goal.

Both models are trained jointly using a combined loss function:

$$L_{\text{joint}} = L_{\text{inv}}(a, \hat{a}) + \lambda L_{\text{fwd}}(s, \hat{s}), \quad (52)$$

where L_{inv} is typically a cross-entropy loss over predicted actions, L_{fwd} is a regression loss on predicted state representations, and λ balances the contribution of the two terms.

8.6.2 Greedy Planning

When a single action is insufficient to reach a desired goal image, the agent uses a **greedy planning strategy**. At each step, the inverse model predicts the best immediate action to move the current image I_t closer to the goal image I_g . This process is repeated iteratively until the target configuration is reached.

- **Advantages:** The approach requires minimal human supervision and avoids the computational overhead of full image reconstruction.
- **Limitations:** Inverse models alone cannot perform long-term planning, since they focus primarily on action prediction and may ignore important environmental structure that is not directly tied to immediate actions.

8.7 Exemplar Models for Exploration (EX^2)

Many exploration strategies in Reinforcement Learning rely on generative models to predict future states or on count-based methods to estimate visit frequency in a state. However, these approaches often struggle in environments with complex state representations. The **Exemplar Models for Exploration (EX^2)** framework addresses this challenge by avoiding explicit density estimation or state modeling altogether, and instead relies solely on **discriminative models** to assess unfamiliar states. Conceptually, **the interaction between models in EX^2 resembles a Generative Adversarial Network (GAN)**, although the relationship is cooperative rather than adversarial.

Given a dataset:

$$\mathcal{X} = \{x_1, \dots, x_n\},$$

typically drawn from the replay buffer, the framework trains a set of exemplar discriminators

$$\{D_{x_1}, \dots, D_{x_n}\}.$$

- **Discriminator Training:** Each discriminator D_{x_i} is trained to distinguish its specific exemplar sample x_i from all other samples in the dataset.

- **Data Balancing:** To approximate the underlying state distribution, the training set for each discriminator is balanced by oversampling the exemplar x_i while drawing multiple negative samples from the empirical distribution $P_{\mathcal{X}}(x)$.

The key intuition behind EX² is that **a state is considered important when it is clearly different from previously observed states**. If a discriminator can easily tell an exemplar state x_i apart from other states in the replay buffer, then that state is rare and therefore highly novel. Such states are prioritized for exploration.

8.7.1 Architectural Building Blocks

Training a separate discriminator for every state in a large replay buffer is computationally impractical. EX² addresses this using two architectural optimizations:

1. **Shared Feature Layers:** All discriminators share early network layers (e.g., convolutional or fully connected layers) to learn general features, while only the final output layers remain exemplar-specific.
2. **Shared Discrimination:** A single discriminator can be trained with multiple labels, enabling it to handle many exemplars simultaneously instead of maintaining one network per state.

The signal produced by the discriminators is incorporated into the reinforcement learning objective as an intrinsic reward:

$$R'(s, a) = R(s, a) + \beta f(D_s(s)), \quad (53)$$

where $D_s(s)$ represents the discriminator’s confidence that state s is unique, β controls the influence of the intrinsic reward, and f is a scaling function. During training, the agent samples trajectories, updates the exemplar discriminators using those samples, computes the reward R' , and then updates the policy.

8.8 Curiosity-Driven Exploration (ICM)

In many reinforcement learning problems, extrinsic rewards (r_e) provided by the environment are extremely sparse, making it unlikely for an agent to encounter them through random exploration. The **Intrinsic Curiosity Module (ICM)** addresses this issue by providing an internal reward signal (r_i) based on curiosity, motivating the agent to explore even in the lack of external feedback.

8.8.1 Architectural Building Blocks

ICM generates intrinsic rewards by focusing only on parts of the environment that the agent’s actions can influence. It consists of two main sub-modules:

- **Inverse Model:** This model takes the encoded features of the current state $\phi(s_t)$ and the next state $\phi(s_{t+1})$, and predicts the action $\hat{a}_t$ that caused the transition:

$$\hat{a}_t = g(\phi(s_t), \phi(s_{t+1}); \theta_I).$$

By learning to predict the agent’s actions, the inverse model forces the feature encoder ϕ to represent only controllable aspects of the environment.

- **Forward Model:** This model predicts the future feature representation based on the current encoded state and the executed action:

$$\hat{\phi}(s_{t+1}) = f(\phi(s_t), a_t; \theta_F).$$

It acts as a forward dynamics model in the learned feature space.

The intrinsic reward is defined as the prediction error of the forward model. If the agent cannot accurately predict the effect of its own action in feature space, the transition is considered unfamiliar and therefore interesting:

$$r_i = \frac{\mu}{2} \left\| \hat{\phi}(s_{t+1}) - \phi(s_{t+1}) \right\|_2^2, \quad (54)$$

where $\mu > 0$ is a scaling hyperparameter. The policy maximizes the total reward, defined as the sum of intrinsic and extrinsic rewards:

$$r_t = r_t^i + r_t^e.$$

Example: A key problem is that not every surprising change is worth exploring. Some changes are caused by the agent’s actions, while others are pure noise that the agent cannot control.

Imagine a robot watching a room scene with a TV in the background. Every frame looks new and unpredictable, but the robot cannot influence what appears on the screen. If curiosity were based only on surprise, the robot would waste all its time watching the TV instead of learning useful behaviors. The Intrinsic Curiosity Module (ICM) avoids this by rewarding only changes that depend on the agent’s actions. It learns to focus on parts of the environment that the agent can control (e.g., moving an object) and ignores random or uncontrollable changes (e.g., wind, noise, or background motion).

9 Lecture 9: Transformers in Deep Reinforcement Learning

9.1 Introduction to Transformers

While Recurrent Neural Networks (RNNs), such as LSTMs, have long been the standard for modeling sequential data, they suffer from two major limitations: slow training due to their inherently sequential computation and difficulty capturing very long-term dependencies. The Transformer architecture addresses these issues by abandoning recurrence entirely in favor of attention mechanisms (*“attention is all you need”*).

9.1.1 The Transformer Architecture

The Transformer processes sequences using attention rather than recurrence, which allows it to handle all elements of a sequence in parallel. Its main building blocks are:

- **Scaled Dot-Product Attention:** Each element in the sequence decides which other elements are important to it. This is done using three vectors: **Queries (Q)** which ask what to focus on, **Keys (K)** which describe the available information, and **Values (V)** which contain the actual content. A scaled dot product is used to compute attention weights in a numerically stable way.
- **Multi-Head Attention:** Instead of using a single attention mechanism, the Transformer uses several attention heads in parallel. Each head can focus on a different type of relationship in the sequence, such as short-range or long-range dependencies.
- **Positional Encoding:** Because the Transformer does not process inputs sequentially, it has no built-in notion of order. Positional encodings are added to the input embeddings to provide information about the position of each element in the sequence.

The full architecture is built from two main parts: an **Encoder**, which processes the input sequence and builds a contextual representation, and a **Decoder**, which uses this representation to generate outputs. Residual connections and layer normalization are applied throughout the network to improve training stability.

9.2 BERT: Bidirectional Representations

BERT (**B**idirectional **E**ncoder **R**epresentations from **T**ransformers) is a deep, multi-layered architecture that builds upon the Transformer and GPT frameworks. Unlike previous models such as OpenAI GPT, which use a left-to-right Transformer, or ELMo, which concatenates independent left-to-right and right-to-left LSTMs, BERT is **bidirectional**. This means its representations are jointly conditioned on both left and right context across all layers. To support this structure, BERT’s input representation is the sum of **token**, **segment**, and **position embeddings**.

Rather than being trained for a specific task from the outset, a general BERT model is pre-trained using two fundamental objectives:

1. **Masked Language Modeling (MLM):** In each sequence, 15% of the tokens are randomly selected for prediction. To prevent the model from relying solely on the [MASK] token, the selected tokens are replaced according to the following distribution: 80% of the time with the [MASK] token, 10% of the time with a random word, and 10% of the time they remain unchanged.
2. **Next Sentence Prediction (NSP):** This is a **binary classification task** designed to determine whether one sentence logically follows another in the input sequence. Although the task itself is relatively simple (with an accuracy of approximately 98%), incorporating it during pre-training has been shown to improve performance across a range of downstream tasks.

By utilizing these two pre-training objectives, BERT achieves state-of-the-art results, with **BERT_{LARGE}** significantly outperforming previous models across multiple benchmarks.

9.3 Challenges of Transformers in DRL

While other fields have rapidly migrated to Transformer-based architectures, the Deep Reinforcement Learning (DRL) research community continues to rely almost exclusively on Recurrent Neural Network (RNN) solutions, such as LSTMs. Several fundamental challenges contribute to this slower rate of adoption:

- **High Data Requirements:** Transformers are notoriously data-hungry and typically require a vast amount of data to achieve peak performance compared to traditional architectures.
- **Optimization Complexity:** These models are significantly more difficult to optimize due to having many interacting components. Research indicates they are highly sensitive to hyperparameters, often necessitating complex learning rate schedules or specialized weight initialization strategies to ensure training stability.
- **Input Space vs. Temporal Memory:** Transformers have demonstrated success when utilized for attention over a complex **input space**. However, they generally struggle when relied upon for temporal memory.
- **Failure on Simple Problems:** Despite their theoretical capabilities, empirical studies have shown that Transformers often fail to solve even simple Multi-Armed Bandit and Markov Decision Process (MDP) problems that standard RL algorithms handle with ease.

In their standard form, Transformers are not well suited for Reinforcement Learning problems. Although RL naturally involves sequential data, directly applying Transformers “as is” often leads to instability and poor performance, particularly when long-horizon dependencies and temporal credit assignment are required. As a result, successful applications of Transformers to reinforcement learning typically require adapting the RL formulation to better align with the strengths of Transformer architectures, as will be discussed in later sections. An alternative direction explored early on was to modify the Transformer itself to better suit RL dynamics. The Gated Transformer architecture represents one such attempt to apply Transformers to reinforcement learning in a more conventional manner.

9.4 Gated Transformer-XL (GTrXL)

The **Gated Transformer-XL (GTrXL)** introduces several architectural modifications to address the inherent sensitivity to hyperparameters and the optimization challenges that often cause standard Transformers to fail in Reinforcement Learning environments. These changes are designed to stabilize the training process and improve the model’s capacity for temporal reasoning:

- **Identity Map Reordering (TrXL-I):** In standard Transformer blocks, layer normalization is typically applied between sub-modules. GTrXL reorders this structure by placing the **layer normalization inside each sub-module** (the Multi-Head Attention and Position-Wise MLP blocks) rather than between them. This ensures that the residual skip connection remains a pure identity map, which make a more sequential and Markovian representation of state transitions. This reordered architecture, denoted as **TrXL-I**, has been shown to achieve significant performance improvements on its own.

- **Gating Mechanisms:** To further control the information flow through the network, GTrXL replaces traditional residual connections with specialized gating layers. These gates, such as the **sigmoid-tanh gate**, are functionally similar to those found in LSTMs and GRUs and are used to modulate the interactions between the module’s input, output, and skip connection. The gating mechanism effectively regulates read and write operations, while the skip connections are designed to bypass normalization entirely.

Additionally, GTrXL utilizes **relative positional encoding** to support a much larger contextual horizon, allowing the agent to model dependencies over longer sequences.

9.5 Handling Long Sequences

Standard Transformer architectures, such as BERT, are limited by a fixed context window (typically 512 tokens) because the computational complexity of full self-attention scales quadratically (n^2) with the sequence length. This makes processing very long trajectories computationally intractable and prevents the model from capturing global context in large-scale tasks. To address these challenges, specialized architectures have been developed to minimize the attention mechanism.

9.5.1 Longformer

The core strategy of the **Longformer** is to maintain a manageable attention span (e.g., a 512-token window) while dividing it among several overlapping “perspectives” to increase the effective receptive field. It utilizes three primary attention patterns:

- **Sliding Window Attention:** Similar to standard self-attention, but restricted to a fixed-size window centered around each analyzed token.
- **Dilated Sliding Window:** Inspired by dilated convolutional neural networks, this pattern skips tokens to increase the attention reach without increasing computational cost. This approach can scale to sequences containing tens of thousands of tokens.
- **Global Attention:** Assigned to fixed, pre-defined locations throughout the input. This creates an asymmetric representation in which global tokens attend to the entire sequence, and all tokens in the sequence attend back to them.

To optimize performance, smaller attention windows are typically used in lower layers to capture local information, while larger windows in higher layers are used to capture broader contextual dependencies.

9.5.2 Big-Bird

Developed concurrently with the Longformer and based on a similar intuition, **Big-Bird** aims to scale Transformer models to long sequences by further reducing computational overhead. Its defining feature is the use of **sparse random attention** in combination with sliding-window and global attention mechanisms. This random sparsification allows the model to preserve key theoretical properties of full self-attention while achieving linear complexity, enabling it to process longer sequences than standard Transformers.

While both Big-Bird and Longformer use sparse attention to efficiently handle long sequences, they differ mainly in how this sparsity is applied. Longformer uses a **fixed and structured pattern**: each token attends only to nearby tokens in a sliding window, with a few predefined tokens allowed to attend to the entire sequence. This structure must be designed in advance.

Big-Bird, on the other hand, adds **random attention connections**. In addition to local and global attention, each token also attends to a small random set of other tokens. For example, a word at the beginning of a long document may randomly attend to a word near the end, even if no global token was explicitly defined. These random connections help the model capture long-range dependencies more flexibly, while still keeping the computation efficient.

9.6 Trajectory Transformer

The **Trajectory Transformer (TT)** represents an attempt to solve offline reinforcement learning as a large-scale supervised sequence modeling problem. Rather than relying on traditional reinforcement learning components, this approach treats a trajectory τ consisting of states, actions, and scalar rewards as one continuous sequence of tokens.

- **Discretization:** To handle the continuous nature of many reinforcement learning environments, the model discretizes states, actions, and rewards into discrete tokens. This discretization is typically achieved using one of two methods: uniform discretization (partitioning by value) or quantile discretization (partitioning by data density).
- **Training:** The model architecture follows the GPT training paradigm, utilizing **teacher forcing** to learn the probability of the next token in the sequence. By modeling the joint distribution over all tokens, the Trajectory Transformer is able to predict long horizons without the **compounding errors** that commonly arise in single-step feedforward dynamics models and often lead to physically implausible rollouts.
- **Planning:** For decision-making, the Trajectory Transformer employs a **standard beam-search** procedure. This planning phase mirrors the sampling process used in language models, enabling the agent to identify high-reward trajectories and to **overcome local optima** through a heuristic search over likely high-return sequences.

In evaluations on locomotion tasks, the Trajectory Transformer has demonstrated performance comparable to, and in some cases exceeding, other state-of-the-art offline reinforcement learning algorithms.

9.7 Decision Transformer (DT)

The **Decision Transformer (DT)** represents a paradigm shift by formulating reinforcement learning as a conditional sequence modeling problem, thereby bypassing the need for traditional RL components such as bootstrapping and discount factors. It employs a causal Transformer (specifically a GPT-style architecture) with modality-specific embeddings for states, actions, and rewards, and processes entire trajectories as a single sequence.

Instead of conditioning actions on past rewards, the model uses the **Return-to-Go**, defined as the sum of all future rewards from the current timestep t :

$$\hat{R}_t = \sum_{t'=t}^T r_{t'}.$$

This formulation makes the model **future-conditioned**: instead of choosing actions based on past rewards, the model selects actions that are expected to lead to a desired future outcome. During deployment, the model is given a **target return** $\hat{R}_1$, which is usually set to a high value (for example, expert-level performance). At each timestep, the Transformer predicts the next action that is most likely to achieve this target return. After the action is executed, the received reward is subtracted from the target:

$$\hat{R}_{t+1} = \hat{R}_t - r_t,$$

and the updated sequence is fed back into the model. This process repeats autoregressively while keeping only the most recent K steps in the context window. The self-attention mechanism allows the model to directly connect important states with delayed rewards, even when they are far apart in time. Unlike Q-learning, which propagates reward information step by step using Bellman updates, self-attention considers all timesteps at once. This makes the Decision Transformer particularly effective in environments with sparse or delayed rewards.

Advantages

- **Stable Training:** DT avoids unstable reinforcement learning components such as bootstrapping and off-policy value updates.
- **Scalable:** The model can be trained using standard supervised learning techniques and benefits directly from advances in Transformer architectures.
- **Flexible Behavior:** A single trained model can produce different behaviors by simply changing the target return provided as input.

9.8 Elastic Decision Transformer (EDT)

The **Elastic Decision Transformer (EDT)** is designed to overcome a fundamental limitation of the standard Decision Transformer: the inability to perform **trajectory stitching**. In offline reinforcement learning, stitching refers to the ability to combine segments of different suboptimal trajectories to form a single near-optimal path. Because the original Decision Transformer is trained with a fixed history window K and is biased toward reproducing exact dataset trajectories, it often fails to create such unfamiliar trajectory combinations. The defining feature of EDT is its use of a **variable history length**, which changes depending on how good the current trajectory is:

- **Suboptimal Path (Short History):** When the agent is on a low-value trajectory, reducing the history length increases the variance of its predictions. This enables the model to “forget” suboptimal past behavior and encourages exploration toward a better continuation.
- **Optimal Path (Long History):** When the agent is following a high-value trajectory, maintaining a longer history window reduces prediction variance and provides greater stability, allowing the agent to preserve consistent high-reward behavior.

9.8.1 Architectural Building Blocks

The EDT architecture extends the causal Transformer used in DT by introducing an additional **return predictor** head alongside the standard action prediction head:

- **Maximum Return Prediction ($\tilde{R}$):** The model predicts the maximum achievable return for each possible history length $\tau \in \{1, \dots, T\}$.
- **Expectile Regression:** To estimate upper-bound returns rather than average outcomes, the return predictor is trained using an **expectile regression loss**, denoted $L_{\max}$, which prioritizes high-return trajectories.

During deployment, EDT does not rely on a fixed context window. Instead, at each timestep t , it performs an adaptive search over possible history lengths:

1. Cut the current trajectory to multiple candidate lengths τ within the search space.
2. Use the EDT to estimate the maximum return $\tilde{R}_t^\tau$ for each shortened sequence.
3. Select the **optimal history length**

$$\tau^* = \arg \max_{\tau} \tilde{R}_t^\tau,$$

which promises the highest future return.

4. Predict the next action using only the sequence truncated to τ^* .

9.9 Online Decision Transformer (ODT)

The **Online Decision Transformer (ODT)** extends the standard Decision Transformer by allowing **offline-to-online fine-tuning**. While the original DT is limited by the quality of a fixed offline dataset, ODT lets the agent continue learning through interaction with the environment and discover better trajectories.

9.9.1 Stochastic Policy and Max-Entropy Exploration

Unlike the original DT, which is often deterministic, ODT models the policy π_θ as a **stochastic distribution** (for example, a Gaussian). This helps the agent explore different actions instead of always choosing the same one.

- **Sequence-level Entropy:** Entropy is **computed over a sequence** of length K instead of a single timestep, encouraging diverse behavior across several actions.
- **Lagrangian Constraint:** Exploration is maintained by forcing the average entropy $H_\tau(\theta)$ to stay above a target value β :

$$\min_{\theta} J(\theta) \quad \text{s.t.} \quad H_\tau(\theta) \geq \beta. \quad (55)$$

This prevents the policy from collapsing into repetitive behavior.

9.9.2 Trajectory Replay Buffer

Standard RL replay buffers store single transitions (s_t, a_t, r_t, s_{t+1}) , which breaks the order of events over time. Since Transformers rely on sequences, ODT uses a **trajectory replay buffer** that stores full episodes. During training, the model samples continuous segments of length K from these trajectories, similar to training in language models. After offline pre-training, the buffer is initialized with the top N trajectories from the dataset. As the agent collects new episodes online, they are added to the buffer while older ones are removed using FIFO (first-in-first-out).

Example: Instead of learning from isolated steps like “move forward”, the model learns from complete motion sequences, preserving long-term dependencies between actions and rewards, which is better to handle the credit assignment problem.

9.9.3 Hindsight Return Relabeling

During online interaction, the agent is conditioned on an intended return g_{online} . In practice, the return actually achieved may be different. This mismatch can confuse the model because the return tokens no longer match what really happened. **Hindsight return relabeling** fixes this by updating the return-to-go tokens to match the **actual rewards obtained**. The corrected trajectories are then used for fine-tuning.

Example: If the agent aimed for a return of 100 but achieved only 70, the trajectory is relabelled with return values consistent with 70 so the model learns from real experience.

ODT combines offline pre-training and online fine-tuning in a single framework and achieves high sample efficiency. In environments such as *Hopper*, ODT performs better than the offline Decision Transformer and shows fine-tuning improvements of about $9\times$ compared to methods such as Implicit Q-Learning (IQL).

9.10 Safe and Multi-Game Transformers

9.10.1 Constrained Decision Transformer (CDT)

The **Constrained Decision Transformer (CDT)** addresses the challenge of learning safe policies from offline datasets, where the agent must satisfy constraints during both training and deployment without online interaction. This problem is formulated as a **Constrained Markov Decision Process (CMDP)** optimization:

$$\max \mathbb{E}[R] \quad \text{s.t.} \quad \mathbb{E}[C] \leq \tau,$$

where R is the cumulative reward, C is the cumulative cost associated with constraint violations, and τ is a user-defined cost threshold.

- **Cost Conditioning:** CDT extends the standard DT token sequence by introducing a **cost token** C , representing the remaining safety budget available to the agent. During deployment, both the target return and cost budget are updated at each timestep based on observed feedback:

$$R_{t+1} = R_t - r_t, \quad C_{t+1} = C_t - c_t.$$

- **Stochastic Gaussian Policy:** To prevent policy collapse and avoid the systematic bias of deterministic policies, which can lead to unwanted distribution actions, CDT employs a stochastic Gaussian policy. This formulation enables a more diverse action space and improves robustness under constraint satisfaction.
- **Zero-Shot Adaptation:** A key innovation of CDT is its ability to generalize to previously unseen cost thresholds τ . By modifying the target cost at deployment, a single trained model can automatically adapt to different safety requirements without retraining.

Example: Consider an autonomous delivery robot that must reach a destination while avoiding collisions. The reward encourages faster arrival, while a cost is assigned whenever the robot drives too close to obstacles. By conditioning the model on both a target return and a cost budget, CDT can generate different behaviors: a low cost budget produces safer and slower trajectories, while a higher cost budget allows more aggressive but riskier navigation. This enables the same trained model to adapt to different safety requirements without retraining.

9.10.2 Multi-Game Decision Transformer

The **Multi-Game Decision Transformer** studies whether a single Transformer model can learn to play many different games using the same shared parameters. Instead of training a separate model for each environment, one model is trained on data from many games. Experiments show that this approach can reach, on average, about **126% of human-level performance**.

- **Sequence and Patch Representation:** High-dimensional observations (such as images) are divided into M smaller image patches at each timestep. The trajectory is represented as

$$\langle \dots, o_t^1, \dots, o_t^M, \hat{R}_t, a_t, r_t, \dots \rangle.$$

The immediate reward token r_t is included explicitly, which helps the model learn from short-term outcomes and not only from the return-to-go.

Example: In Atari games, the screen image is split into patches. The model can focus separately on different regions (e.g., enemies, player, obstacles), similar to how vision Transformers process images.

- **Expert Action Inference:** The model is trained on trajectories from many games, including both expert and non-expert demonstrations. During deployment, the action probabilities are adjusted so that actions similar to expert behavior are preferred.

Example: If two actions are possible but one appears more often in expert gameplay, the model gives higher probability to that action.

- **Expert Fine-Tuning:** Performance can be further improved by fine-tuning the model using additional high-quality trajectories collected from strong Decision Transformer agents.

9.11 Feedback Decision Transformer (FDT)

The **Feedback Decision Transformer (FDT)** improves the standard Decision Transformer by adding external feedback during training, which is basically **Reward Shaping**. Offline datasets often contain imperfect or limited data, and collecting expert demonstrations is expensive. FDT uses feedback from an **Oracle** (for example, a human or a pre-trained agent) to help the model focus on the most important decisions.

To simplify the state space, FDT groups similar states into **state cells**:

1. **Clustering:** States from training trajectories are grouped according to similarity.

Example In a game, states where the agent is close to an obstacle may be grouped into the same cell.

2. **Importance Identification:** The model finds critical states where the chosen action strongly affects future rewards.

Example: In *Breakout*, a state where the ball is close to the paddle is more important than one where the ball is far away.

3. **Cell-Level Sorting:** Each cell i receives an importance score:

$$\text{Diff}_i = \text{AvgRTG}_i^{\max} - \text{AvgRTG}_i^{\min}. \quad (56)$$

Only the top X most important cells are selected for feedback.

The selected cells are evaluated by the Oracle:

- **Labeling:** Actions approved by the Oracle receive positive feedback ($f = 1$); otherwise they receive negative feedback ($f = 0$).
- **Alternative Actions:** For negative examples, the system stores a **recommended alternative action (RCA)** the action in the same cell that achieved the highest return.
- **Augmented Representation:** Each sample is represented as

$$(s_t, a_t, r_t, f, \text{rca}_t),$$

where f indicates the feedback and rca_t is the suggested alternative action.

FDT uses a combined loss:

$$\mathcal{L}_{\text{FDT}}(\theta) = \mathcal{L}_{\text{DT}} + \lambda_F \mathcal{L}_{\text{PF}} + \lambda_F \mathcal{L}_{\text{NF}}. \quad (57)$$

The model is encouraged to repeat actions with positive feedback and replace poor actions with the recommended alternatives.

Example: If the Oracle marks an action as bad (e.g., moving left into danger), the model is trained to prefer the suggested alternative action (e.g., moving right). Empirical results show that FDT performs better than the standard Decision Transformer, especially when generalizing beyond the original training data.

10 Lecture 10: Model-Based Approaches and Gradient-Free Learning

10.1 Introduction to Model-Based Learning

Model-based reinforcement learning focuses on explicitly modeling the underlying dynamics of an environment. By combining this learned dynamics model with a known cost or reward function, an agent can perform optimal planning. One of the primary advantages of this approach is its extreme sample efficiency compared to model-free methods. For example, while a model-free approach might require 10^8 steps (approximately 15 days of real-time interaction) to master a task, model-based methods can often achieve similar performance in only 10^5 steps, or roughly 3 hours. Despite this efficiency, these methods face several challenges:

- **Optimization Complexity:** Finding an optimal plan through a learned model is often significantly more difficult than simple policy execution.
- **Model Bias:** This occurs when an agent assumes its currently learned (and likely inaccurate) dynamics are sufficiently correct. This leads the agent to make arbitrary decisions with full certainty, potentially based on incorrect model assumptions of the environment's behavior.
- **Modeling Uncertainty:** To mitigate bias, agents may use probabilistic approximators (such as Gaussian Processes) to model uncertainty. While Gaussian Processes are highly data-efficient, they struggle with non-smooth dynamics and high-dimensional data, whereas Neural Networks are expressive but require more data to be effective.

10.1.1 Optimization Objective and Gradients

The goal in model-based learning is to minimize the total cost accumulated over a trajectory of length T :

$$\min_{a_1, \dots, a_T} \sum_{t=1}^T c(s_t, a_t), \quad \text{where } s_t = f(s_{t-1}, a_{t-1}) \quad (58)$$

Here, $c(s_t, a_t)$ is the cost at each step, and f describes how the state changes according to the current state and action. To optimize this objective using gradient-based methods, the agent needs the derivatives of both the dynamics and the cost:

- $\frac{df}{ds_t}$ and $\frac{df}{da_t}$: how the next state changes when the state or action changes.
- $\frac{dc}{ds_t}$ and $\frac{dc}{da_t}$: how the cost changes with respect to the state or action.

These gradients allow the agent to update through the dynamics model and improve its actions or policy.

10.1.2 Modeling the Environment

When the environment dynamics are unknown, the agent must learn an approximation of the transition function $f(s, a)$. Choosing the right model involves trade-offs between data efficiency, flexibility, and computational cost.

- **Gaussian Processes (GP)**: Probabilistic models that capture uncertainty in the dynamics. They are very data-efficient but become slow for large or high-dimensional datasets.
- **Neural Networks (NN)**: Highly expressive models that work well in complex environments and large datasets, but usually require many training samples.
- **Gaussian Mixture Models (GMM)**: These divide the state space into multiple regions and learn local models in each region, which can be useful for modeling physical systems with different local behaviors.

10.2 Evolution of Model-Based Learning Algorithms

Model-based reinforcement learning has evolved through several versions to improve sample efficiency, robustness, and optimization stability. The main approaches are summarized below, together with simple intuitive examples.

- **Version 1 (Simple MBL)**: Learn a dynamics model from collected data and use it to plan actions. The agent runs a base policy π_0 , collects transitions $D = \{s, a, s'\}$, learns a dynamics model $f(s, a)$ using mean squared error (MSE), and backpropagates through the model to choose actions.
Example: A robot moves randomly in a room, learns how actions change its position, and then plans a path using this learned model.
Pros: Simple and easy to implement with no iterative re-training.
Cons: Small modeling errors accumulate, so the robot may drift into regions where the model is inaccurate.
- **Version 2 (Dagger-style)**: Repeatedly update the model using new data generated by the agent itself. After planning, the agent executes actions, adds the new data to D , and re-trains the dynamics model.
Example: The robot follows a planned path, records where it actually ended up, and updates the model to reduce future mistakes.
Pros: Reduces distribution mismatch by training on states that are actually visited.
Cons: Plans are still open-loop, so performance may degrade in noisy or changing environments.

- **Version 3 (Model Predictive Control – MPC):** Re-plan continuously to correct modeling errors online. The agent plans a sequence of actions but executes only the first action, observes the new state, and re-plans every step (or every N steps).
Example: A self-driving car recomputes its trajectory every second instead of following one long fixed plan.
Pros: Robust to disturbances and model errors because it continuously corrects itself.
Cons: Computationally expensive since planning must run repeatedly during execution.
- **Version 4 (Direct Policy Optimization):** Optimize the policy directly by backpropagating through the learned dynamics model. Instead of planning actions, the agent updates a policy $\pi_\theta(a_t | s_t)$ using gradients through the model.
Example: A simulated robot learns a neural network controller by imagining future trajectories inside the learned model and updating the policy directly.
Pros: Fast at runtime because only the policy is evaluated.
Cons: Can be numerically unstable due to vanishing or exploding gradients when trajectories are long.

10.2.1 The PILCO Algorithm

PILCO (Probabilistic Inference for Learning Control) is a data-efficient model-based method that learns a probabilistic dynamics model **using Gaussian Processes** and performs policy optimization through the learned model. Unlike deterministic approaches, PILCO represents the transition dynamics as a probability distribution:

$$p(s_t | s_{t-1}, a_{t-1}) = \mathcal{N}(s_t | \mu_t, \Sigma_t),$$

which explicitly models uncertainty in predictions. PILCO optimizes the policy by propagating gradients through the learned dynamics model. However, the main difference is that PILCO uses a **probabilistic** dynamics model (Gaussian Processes), allowing it to account for uncertainty during policy optimization instead of relying on a purely deterministic model. The agent first learns a probabilistic model of how the system behaves, then improves the policy by predicting future outcomes inside this model, updating the policy parameters to reduce expected cost. Because uncertainty is modeled explicitly, PILCO achieves very high data efficiency and can learn control tasks with relatively few real-world interactions.

10.3 Global vs. Local Models

Standard model-based Reinforcement Learning originally relied on **global models**, which attempt to approximate a single dynamics function $f(s, a)$ across the entire state space. However, these approaches face significant practical limitations:

- **Inefficient Optimization:** Global models tend to seek out and improve regions where their performance is worst. This often wastes computational effort on areas that are irrelevant to the specific task or that the agent may never visit during optimal execution.
- **Excessive Complexity:** In many environments, the dynamics model required to map the entire world is significantly more complex and harder to learn than the policy needed to solve the task.

To mitigate these issues, **local models** focus on accurately describing the system only around the agent’s **current operating region**, instead of trying to learn one model that works everywhere. The main idea is that, for selecting the next action, the agent mainly needs an accurate model near its current trajectory, not across the entire state space. Local models therefore build small and simple approximations that are updated as the agent moves through the environment and guide decisions by focusing on the immediate neighborhood of the current trajectory:

- **Linearization:** The dynamics are approximated using a first-order Taylor expansion at each time step. This creates a local linear model that is easier to optimize using methods such as LQR or iLQR.
- **Localized Sampling:** Data is collected mainly around the states the agent currently visits. This ensures that the learned model is accurate where decisions are actually being made.

Example: A robot arm moving toward a target does not need to model every possible arm position. Instead, it approximates the dynamics only around its current motion and computes the next action using this local approximation. As the arm moves, the local model is updated again.

10.4 Linear Quadratic Regulator (LQR)

When the system dynamics are linear and the cost function is quadratic, the **Linear Quadratic Regulator (LQR)** provides an exact solution for continuous control problems. Instead of maximizing rewards, LQR minimizes a cost function.

- **Linear Dynamics:**

$$x_{t+1} = Ax_t + Bu_t$$

where x is the state (equivalent to s) and u is the input (equivalent to a).

- **Quadratic Cost:**

$$g(x_t, u_t) = x_t^T Q x_t + u_t^T R u_t$$

where Q penalizes state deviation from the target and R penalizes large control inputs.

LQR naturally encourages fast convergence because remaining far from the target accumulates higher cost. The goal is to compute a gain matrix K such that the optimal control law is

$$u = -Kx.$$

The algorithm operates in two stages: a **backward recursion** that computes optimal gains starting from the final timestep, followed by a **forward recursion** that applies the resulting actions. Its computational complexity is $O(Tn^3)$, which can be expensive for high-dimensional systems. If the system includes zero-mean Gaussian noise, the optimal policy remains unchanged, meaning the noise can be ignored during planning.

10.4.1 Iterative LQR (iLQR)

For non-linear systems, **iLQR** extends LQR by locally approximating the system around a nominal trajectory using Taylor expansions:

1. **Linearize Dynamics:** Perform a **first-order Taylor expansion** of the transition function $f(x_t, u_t)$.
2. **Quadraticize Cost:** Perform a **second-order Taylor expansion** of the cost function $c(x_t, u_t)$.
3. **Optimization:** Run an LQR backward pass on deviation terms $(\delta x_t, \delta u_t)$, where

$$\delta x_t = x_t - \hat{x}_t,$$

followed by a forward pass using the **true non-linear dynamics** to update the trajectory.

10.5 Model Predictive Control (MPC)

While iLQR is effective for planning, it is inherently an open-loop method once the trajectory is fixed, making it sensitive to disturbances, stochastic noise (e.g., wind), and modeling errors. To improve robustness, it is commonly combined with **Model Predictive Control (MPC)**.

In MPC, the agent does not execute the full planned action sequence. Instead, it repeatedly follows the following cycle:

1. Observe the current state x_t .
2. Solve the optimization problem (often using iLQR) from the current timestep to the horizon T .
3. Execute **only the first planned action** and discard the remaining plan.
4. Observe the new state and **re-plan** immediately.

This repeated re-planning process is called **receding horizon control**. It allows the controller to correct its actions at every step, making the system much more robust to disturbances and modeling inaccuracies.

10.6 Guided Policy Search (GPS)

Guided Policy Search (GPS) connects trajectory optimization with policy learning in systems where dynamics are unknown. The method alternates between two stages:

1. **Trajectory Optimization:** Uses iterative-LQG (a variant of iLQR) to compute locally optimal trajectories. Since dynamics are unknown, samples (x, u, x') are used to fit local linear models.
2. **Policy Optimization:** A high-capacity deep neural network (DNN) policy is trained via supervised learning to imitate these optimized trajectories.

To keep the learned policy close to trusted trajectories, GPS minimizes the **KL-divergence** (D_{KL}) between the policy-generated trajectory distribution and the optimized trajectory distribution. This prevents the policy from drifting into regions where the local models are unreliable.

10.7 Gradient-Free Methods and Evolutionary Algorithms

When the optimization landscape is rugged (non-convex, discontinuous, noisy, or high-dimensional), gradient-based methods may fail. **Evolutionary Algorithms (EA)** provide a black-box optimization alternative by evolving a population of candidate solutions evaluated through a fitness function.

The population is updated using three main operations:

- **Selection:** Stochastically chooses high-fitness individuals as parents.
- **Mutation:** Applies local random perturbations to generate new candidates.
- **Recombination:** Combines multiple parents to create non-local variations.

Evolutionary algorithms naturally balance exploration (trying new solutions) and exploitation (improving good solutions).

10.8 Covariance Matrix Adaptation Evolution Strategy (CMA-ES)

CMA-ES is an advanced gradient-free optimization method based on two key principles:

- **Maximum Likelihood:** Updates the sampling distribution to increase the probability of generating successful candidates.
- **Covariance Matrix Adaptation:** Updates the covariance matrix to favor directions that previously led to improvement, effectively shaping the search space.

CMA-ES can be viewed as a form of **natural gradient descent** because it considers both gradient information and the curvature of the distribution space through the Fisher information matrix. Although it uses second-order information, it does not rely on backpropagation, making it suitable for non-smooth or noisy optimization problems.

11 Lecture 11: World Models in Deep Reinforcement Learning

11.1 Introduction to World Models

World Models are **learned predictive models that simulate environment dynamics within a latent space**. Instead of interacting directly with the high-dimensional real world for every update, an agent uses a World Model to predict future states given current observations and actions. This enables imagination, allowing the agent to plan trajectories and learn policies through simulation without the need for constant real-world interaction. The primary motivations for this approach include:

- **Sample Efficiency:** World models can achieve a 10–100× improvement in sample efficiency over model-free methods by reducing the necessity for real-world interactions and learning from fewer environment samples.
- **Offline Planning:** Tasks can be solved via simulation, avoiding costly or dangerous real-world rollouts.
- **Credit Assignment:** Gradients can be propagated through imagined trajectories, which aids in more effective policy learning by establishing clearer links between actions and outcomes.
- **Generalization:** These models learn compact latent representations that capture only the features relevant to the task, ignoring redundant environmental details.

11.1.1 Architectural Building Blocks

A standard World Model architecture consists of three core components that allow an agent to transition from high-dimensional environmental input to internal simulation and decision-making:

1. **Perception (Encoder):** This component compresses high-dimensional observations (e.g., raw pixels) into compact **latent representations**. In frameworks like PlaNet and Dreamer, this often involves a representation model $p(s_t \mid s_{t-1}, a_{t-1}, o_t)$ that predicts the current hidden state from the history and new observations.
2. **Dynamics (Prediction):** Often utilizing a recurrent structure (such as an RNN), this component learns the **temporal evolution** of latent states based on taken actions. In hybrid models like the **Recurrent State-Space Model (RSSM)**, this includes both a **deterministic component** (h_t) to capture temporal dependencies and a **stochastic component** (z_t) to represent uncertainty.
3. **Behavior (Policy):** An **Actor-Critic structure** learns optimal actions $q_\phi(a_t \mid s_t)$ and value estimates $v_\psi(s_t)$ entirely within the latent space of the world model.

11.1.2 Information Flow

The information flow in these models facilitates two distinct modes of operation:

1. Standard Processing and Acting

When interacting with the real world, the flow follows a sequential path to anchor the agent’s internal state:

Observations $\rightarrow$ Encoder $\rightarrow$ Latent State $\rightarrow$ Dynamics $\rightarrow$ Predicted Next State.

The model uses an observation decoder $q(o_t \mid s_t)$ as an helping signal during training to ensure the latent state captures visual details necessary for control, though this is often omitted during planning to save computation.

2. Imagination and Policy Refinement

During the imagination phase, the agent “unrolls” trajectories entirely within its internal latent space without environment interaction. The policy selects actions, the reward model $p(r_t \mid s_t)$ predicts immediate gains, and the value model estimates long-term returns.

Because the world model is differentiable, gradient backpropagation can be propagated through these imagined rollouts to refine the policy. This enables highly effective credit assignment, as the agent can directly see how changing an action in the past affects future rewards predicted by the dynamics. This process allows for a 10–100× improvement in sample efficiency compared to traditional model-free methods.

World model architectures can be analyzed along several key design dimensions, each capturing a fundamental modeling choice. The following subsections summarize four major dimensions.

11.2 Dimension 1: Latent Representation Types

The choice of latent representation is the **most fundamental design decision** in world model architecture, as it directly determines the efficiency of **compression, gradient flow, and the long-term stability** of imagined trajectories. All other dimensions are built based on the chosen latent representation.

11.2.1 Continuous Latents

Continuous latents typically utilise a Variational Autoencoder (VAE) framework to compress high-dimensional observations into a smooth, structured vector space. For example, an image of a robotic arm could be encoded into a continuous vector representing joint angles, object positions, and scene layout.

- **Strengths:**

- **Smooth Gradient Flow:** Gradients propagate without interruption through the continuous space during backpropagation.

Example: Small changes in the latent vector correspond to small changes in predicted arm position, making learning stable.

- **Principled Uncertainty:** The model represents uncertainty by learning distributions instead of fixed values, and KL divergence helps keep these distributions stable and well-behaved.

Example: If the robot’s camera view is partially blocked, the model can represent multiple possible object locations rather than committing to one exact value.

- **Continuous Dynamics:** These models are a natural fit for **physics-based environments** where state transitions are smooth.

Example: In a pendulum or robotic joint system, positions change gradually rather than jumping between discrete states. The real world is continuous and not discrete.

- **Weaknesses:**

- **Compression Inefficiency:** Continuous spaces struggle to represent **discrete events** or categorical changes, such as a collision occurring or an object being “picked up”.

Example: The event “gripper closed on object” is binary, but a continuous space must approximate it with thresholded values.

- **Compounding Errors:** Small prediction inaccuracies in the latent space tend to **compound exponentially** when rolled out over long horizons, leading to “drifting” in imagination.

Example: A tiny angle error in a simulated robotic arm can result in the arm being completely misplaced after 50 imagined steps.

11.2.2 Discrete Latents

In this approach, observations are encoded as a combination of discrete tokens sampled from a learned codebook (e.g., VQ-VAE or VQ-VAE-2). Instead of a continuous vector, the state may be represented as a sequence of learned symbolic indices.

- **Strengths:**

- **Superior Compression:** Discrete tokens are efficient at encoding visual information and can represent distinct, structured concepts.

Example: One token might consistently represent “table surface,” another “robot gripper,” and another “object present.”

- **Long-Horizon Stability:** Discrete transitions are significantly more **stable** for long-horizon imagined rollouts than continuous ones.

Example: A token representing “object grasped” remains stable instead of slowly drifting numerically.

- **Natural Structure:** The token-based format is highly compatible with the structure of **language, discrete actions**, and discrete environmental state changes.

Example: This makes integration with transformer-based models easier.

- **Weaknesses:**

- **Quantization Bottleneck:** Information is inevitably lost during the quantization process (mapping to the nearest neighbor token).

Example: Slightly different object positions may map to the same token, losing fine detail.

- **Biased Gradients:** Because discrete sampling is non-differentiable, the model must rely on the **straight-through estimator**, which provides a biased gradient approximation.

11.2.3 Hybrid Approaches (RSSM)

The Recurrent State-Space Model (RSSM) represents a state s_t as a combination of two distinct components to maximize both stability and flexibility. It combines a **deterministic component** (h_t), which is an RNN hidden state capturing long-term temporal dependencies and providing stable memory, with a **stochastic component** (z_t), which consists of random variables used to represent environmental uncertainty. h_t may encode the long-term motion pattern of a robot navigating a room, while z_t may represent uncertainty about whether an object is behind an obstacle.

- **Strengths:**

- **Better Prediction:** The deterministic memory prevents the rapid error accumulation common in purely stochastic models, enabling **better long-horizon imagination**.

Example: The robot’s planned trajectory remains coherent even after many imagined steps.

- **Uncertainty Separation:** This architecture naturally separates **aleatoric uncertainty** (randomness in the environment) from **epistemic uncertainty** (limitations of the model’s knowledge).

Example: The model distinguishes between a slippery floor (inherent randomness) and an unseen obstacle (lack of knowledge).

- **Weaknesses:** This approach is **more complex**, requiring sensitive tuning of the hidden state size, the number of discrete categories, and the balance of the KL loss. More models = More problems.

11.3 Dimension 2: Dynamics Formulation

The method used to train a world model to predict future states significantly impacts the accuracy of long-horizon planning. The common approaches to this formulation:

11.3.1 One-Step Prediction:

This approach focuses on a simple optimization to predict the immediate next state: $L = |s_{t+1} - \hat{s}_{t+1}|^2$. While it is easy to implement and computationally cheap, it suffers from a critical weakness: **exponential error accumulation**. In this scenario, small errors (ϵ) compound over a horizon (H). For example, after 50 steps, the predictions are often completely “broken”.

Example: If a robot predicts its joint angle with a tiny 1° error at each step, after many imagined steps the arm position may be completely wrong. This limitation is why models like **PlaNet** require online Model Predictive Control (MPC) to re-plan every few steps rather than committing to long trajectories.

11.3.2 Multi-Step Prediction:

To improve long-horizon imagination quality, models are trained to predict multiple steps (e.g., 5 to 50) into the future. Unlike one-step methods, these models utilize **actual model predictions** during training rather than ground-truth states. This approach enables **behavior learning purely in imagination** and reduces the need for constant re-planning. The model learns to maintain stable robot motion over 20 imagined steps, not just one.

Example: Techniques like **scheduled sampling** (gradually shifting from ground-truth to model predictions) and auxiliary losses are used to stabilize training. **Dreamer** typically uses 15-step imagination, while **DreamerV3** can optimize for 50+ steps.

11.3.3 Implicit Dynamics (Search-Based):

Instead of explicitly predicting s_{t+1} , these models learn a representation where simple search or retrieval methods (like nearest neighbor) identify similar dynamics from stored trajectories.

- **Strengths:** This avoids sequential error accumulation and can naturally handle **multi-modality** (multiple possible futures). Likely to perform well in Zero-Shot problems on familiar environments.

Example: If a robot pushes a block and multiple outcomes are possible, the model can retrieve different past trajectories rather than averaging them into an unrealistic prediction.

- **Weaknesses:** It is computationally expensive, requiring a search at every decision point, and may struggle to generalize to situations not stored in memory (like scenarios we have not seen before).

11.3.4 Latent vs. Pixel Space:

The training space choice involves a trade-off between perception grounding and efficiency:

- **Latent Space:** Dominates modern RL (e.g., PlaNet, Dreamer) because it is faster, more stable, and ignores task-irrelevant background details. RL typically requires **task-relevant features** rather than pixel-perfect reconstruction.

Example: The model only tracks object positions and robot pose instead of wall color or lighting.

- **Pixel Space:** Predicts future image pixels directly, making it explainable and grounded. However, optimization is difficult because pixel space is **high-dimensional and redundant**.

Example: The model must predict every pixel of a 128×128 image, even background textures that do not affect the task.

Latent spaces are preferred in RL because they are more computationally efficient, and RL typically requires only task-relevant features rather than pixel-perfect reconstruction.

11.4 Dimension 3: Planning and Behavior Learning Strategy

Once a world model is trained, the agent must determine how to use that internal simulation to make decisions. There are two primary paradigms for this process: **Online Planning** and **Latent Imagination**.

11.4.1 Online Planning (Model Predictive Control)

Online Planning, specifically Model Predictive Control (MPC), treats planning as the policy itself without learning a separate behavior function. This approach is mainly used in PlaNet. At each decision step, the agent solves an optimization problem to find the action sequence that maximizes the predicted cumulative reward:

$$\arg \max_a \sum_{t=0}^T \gamma^t r(s_t, a_t).$$

Common optimization techniques include the Cross-Entropy Method (CEM), shooting methods, or gradient-based planning. The agent executes only the **first action** of the optimized sequence, observes the new state,

and re-plans from scratch at the next step.

Example: The robot plans 10 steps ahead but executes only the first move, then replans after receiving new sensor data.

- **Strengths:** This method is highly reactive, allowing the agent to adapt immediately to unexpected observations. It works effectively even with imperfect models because the constant re-planning compensates for shorter horizons and accumulated errors.
- **Weaknesses:** It is computationally expensive and slow at test time, as the agent must evaluate hundreds of trajectory samples (e.g., PlaNet searches 200 sequences per step) for every single decision. Additionally, optimization methods like CEM struggle with sparse rewards.

Example: In real-time robotics, repeatedly evaluating many trajectories can become too slow for high-speed control.

11.4.2 Latent Imagination (Learned Policy)

Latent Imagination shifts the computational burden from test-time planning to training-time policy optimization. This approach is the foundation of the Dreamer architecture. The agent learns an Actor-Critic architecture consisting of an Action model $q_\phi(a_t | s_t)$ and a Value model $v_\psi(s_t)$ entirely within the world model’s latent space. Instead of searching for actions online, the agent learns a policy that directly outputs actions from imagined states.

After training the world model on replay data, the model is “frozen”. The agent then “dreams” by rolling out imagined trajectories where the actor samples actions, the dynamics model predicts states, and the reward model predicts gains.

Example: The agent practices thousands of simulated trajectories internally without interacting with the real environment.

Unlike MPC, Dreamer optimizes the policy by propagating analytic gradients of multi-step value estimates back through the differentiable latent dynamics. The value model provides bootstrapping, accounting for rewards beyond the finite imagination horizon H .

- **Strengths:** This method is highly scalable and sample-efficient, capable of learning from thousands of imagined steps for every real-world interaction. It handles sparse rewards effectively through long-horizon credit assignment.
 - **Weaknesses:** It is sensitive to model errors; if the world model is inaccurate, the policy will be optimized on a “distributional shift” that does not exist in reality. It also requires a “cold start” period to learn a baseline world model before behavior learning can succeed.
- Example:** A robot can learn mostly from internal simulation while requiring only occasional real-world data collection.
- Example:** If the imagined physics is wrong, the robot may learn actions that fail when executed in the real environment.

11.4.3 Hybrid Approaches

Hybrid approaches combine imagination-based learning with real-world refinement, aiming to balance sample efficiency with robustness to model errors. The agent first trains behavior inside the world model to obtain a rough policy, and then uses real environment interactions to refine and correct this policy. This reduces exposure to model inaccuracies while retaining the efficiency benefits of imagination-based learning.

Example: A robot first learns navigation strategies in imagined rollouts and later fine-tunes the policy using real sensor data.

In this setting, both the world model and the policy are updated continuously as new data is collected. As the world model improves, the policy benefits from increasingly accurate predictions.

Example: During deployment, the robot keeps improving its dynamics model while simultaneously updating its policy based on newly observed behavior.

Hybrid methods iterate between imagination-based learning and real-world correction to combine scalability with reliability.

11.5 Dimension 4: World Model Training Objectives

The loss function chosen to optimize a world model is critical, as it shapes exactly what the model learns to predict and prioritize within its latent space. There are four primary objectives used in modern research:

11.5.1 Maximum Likelihood Estimation (MLE) / Reconstruction Loss:

This is the standard probabilistic approach, typically minimizing the negative log-likelihood of observations:

$$L = -\log p(x_t | z_t) \quad (59)$$

In practice, this often involves image reconstruction paired with reward and value prediction losses.

Example: The model reconstructs the next camera image of a robot scene and compares it to the real image to learn meaningful representations. Used in PlaNet and early Dreamer implementations

- **Strengths:** It is principled, stable, and highly **interpretable**, as researchers can visualize reconstructed images to “see” what the model knows.
- **Weaknesses:** The model often wastes capacity on imperceptible or **task-irrelevant details** (e.g., complex background textures) that do not impact decision-making. This objective is often misaligned with actual downstream RL performance.

Example: The model might learn to perfectly reconstruct wall textures even though they do not matter for control.

11.5.2 Task-Aligned Loss:

Rather than reconstructing every pixel, this objective optimizes directly for downstream task performance using verifiable signals. It utilizes signals such as **action prediction** (inferring the action taken from state transitions) or contrastive metrics to ensure predicted next states remain similar to actual successors in a task-relevant way.

Example: Instead of predicting pixels, the model learns representations that help predict whether a robot action successfully grasped an object. Used in RLVR-World.

- **Strengths:** It minimizes the “task-performance gap” and forces the model to focus exclusively on **decision-relevant features**.
- **Weaknesses:** It requires a clear task definition or access to rewards, making it difficult to use for pre-training on unlabeled data.

11.5.3 Uncertainty Minimization:

This approach focuses on quantifying what the model does not know to guide the agent’s behavior. The agent trains an **ensemble** of world models and minimizes their **disagreement** (variance in predictions):

$$L = \text{Var}_i \hat{s}_i(s_t, a_t) \quad (60)$$

Example: If different models predict very different future states, the agent interprets this region as uncertain and explores it more. Used in PETS.

- **Strengths:** It provides a natural signal for **exploration**, as the agent is incentivized to visit uncertain regions to reduce ensemble disagreement. It also makes the system more robust to individual model failures.
- **Weaknesses:** It is computationally expensive, often requiring $3\times$ to $5\times$ the processing power to maintain multiple models, and may fail to capture inherent environmental randomness (**aleatoric uncertainty**).

11.5.4 Contrastive / Self-Supervised Learning:

This emerging approach learns dynamics representations without any explicit pixel reconstruction. The loss ensures that embeddings of a state s_t and its true successor s_{t+1} are mathematically “close,” while embeddings of s_t and random, unrelated states are kept “distant.”

Example: Representations of consecutive robot states are pulled closer together, while unrelated states from different trajectories are pushed apart.

- **Strengths:** It is **highly scalable** because it removes the need for a decoder, allowing for much larger models. It also avoids learning spurious pixel-level correlations.
- **Weaknesses:** It is less interpretable, as there are no images to visualize, and the system’s success is highly dependent on a carefully designed contrastive loss.

11.6 PlaNet (Deep Planning Network)

PlaNet is designed to solve continuous control tasks from high-dimensional pixel observations in environments where dynamics are unknown. It addresses the challenge of data-efficient control despite partial observability and the exponential accumulation of errors inherent in multi-step predictions.

Unlike previous models that might plan in high-dimensional image space, PlaNet plans purely within a compact **latent space**. This allows the agent to evaluate thousands of candidate action sequences without the computational burden of rendering predicted images.

To maintain prediction consistency over time, PlaNet introduces **latent overshooting**. This is a multi-step variational objective that trains the model to ensure consistency between multi-step latent predictions using KL-divergence terms in the latent space, thereby avoiding expensive multi-step image decoding.

PlaNet utilizes **Model Predictive Control (MPC)** for decision-making. At each timestep, the agent:

1. Infers a belief over its current latent state from the history of observations.
2. Uses the **Cross-Entropy Method (CEM)** to search for an action sequence that maximizes the predicted cumulative reward under the learned dynamics.
3. Executes only the first action and re-plans at the next step.

11.6.1 Architectural Building Blocks

The architecture can be understood as a sequence of components working together:

1. **Transition Model:** The core dynamics model predicts how the latent state evolves over time:

$$p(s_t \mid s_{t-1}, a_{t-1})$$

It models how the previous latent state and action determine the next latent state.

2. **Deterministic Memory (h_t):** A deterministic RNN hidden state is used to capture temporal context and long-term dependencies across time steps. This provides stable memory for planning.
3. **Stochastic Latent Variables (z_t):** Stochastic latents represent uncertainty and allow the model to capture multiple possible future outcomes.
4. **Observation Model:** An observation decoder

$$p(o_t \mid h_t, s_t)$$

is trained to reconstruct images from the latent representation. This provides a strong training signal and ensures that the latent state preserves important visual information.

5. **Planning Usage:** During planning, only the latent dynamics are used. The observation model is omitted to avoid the computational cost of image reconstruction.

11.7 Dreamer

Dreamer represents a paradigm shift from PlaNet by replacing computationally expensive online MPC with a **learned behavior policy** trained entirely within “latent imagination”.

Dreamer uses **Actor-Critic in Imagination** by learning both an **Action model** $q_\phi(a_t | s_t)$ and a **Value model** $v_\psi(s_t)$ within the latent space. It optimizes the policy by propagating analytic gradients of multi-step value estimates back through the differentiable latent dynamics. The Two-Phase Learning Cycle is as follows:

- **Phase 1: World Model Learning:** The encoder, dynamics, and reward models are trained on trajectories sampled from a replay dataset D .
- **Phase 2: Behavior Learning:** The world model is frozen, and the actor and critic are optimized through imagined rollouts. The value model provides **bootstrapping** beyond the finite imagination horizon H , allowing the agent to account for rewards beyond the immediate rollout.

During the imagination phase, trajectories start from latent states found in the replay dataset. The model then rolls forward using the transition model and the current policy to define a fully observed latent MDP. This cycle of learning continues iteratively:

learning a model from experience $\rightarrow$ learning behavior via imagination $\rightarrow$ collecting more real-world data

Because behavior learning happens through forward passes and backpropagation in a parallelizable latent space, Dreamer is highly scalable. It can learn from thousands of imagined steps for every single real environment interaction, achieving significantly higher sample efficiency than model-free methods.